

[PSI3471 aula 10. Início.]

Estou supondo que a maioria dos alunos cursaram PSI3471 (Fundamentos de Sistemas Eletrônicos Inteligentes). Porém, como pode haver alunos que não cursaram PSI3471, nesta primeira aula vamos recordar os conceitos mais importantes. Além disso, vamos estudar com mais detalhes alguns conceitos que não foram ensinados com ênfase em PSI3471.

Rede neural densa em Keras sobre TensorFlow ou PyTorch

1. Keras, Tensorflow, PyTorch e Colab

A partir desta aula, vamos trabalhar com aprendizado profundo, usando redes neurais profundas (convolucionais ou transformers). Trabalharemos em linguagem Python, usando bibliotecas Keras sobre TensorFlow ou PyTorch ou JAX, juntamente com OpenCV.

Nota: Alguns programas foram traduzidos para PyTorch e estão nos anexos das versões antigas desta apostila.

Lançada em 2015, a Keras é uma biblioteca de código aberto em Python desenvolvida para simplificar a construção e o treinamento de modelos de aprendizado profundo. Destaca-se por sua interface amigável e de alto nível, que viabiliza a prototipagem rápida e intuitiva. Contudo, essa mesma simplicidade pode se tornar uma limitação ao desenvolver arquiteturas altamente personalizadas ou fora dos padrões convencionais.

Originalmente, Keras foi desenvolvida para ser usada em cima do TensorFlow, mas com o tempo ganhou suporte para outras *backends*, como o PyTorch e JAX. Neste curso, vamos usar Keras com *backend* TensorFlow.

TensorFlow é uma biblioteca de código aberto para aprendizado de máquina, lançado em 2015. Foi desenvolvida pela equipe Google Brain e é usada tanto para a pesquisa quanto produção na Google. Escrever aplicações de aprendizado de máquina diretamente em TensorFlow costuma ser uma tarefa árdua.

Nota: O que é um tensor (de TensorFlow)? Em aprendizado de máquina, tensor é a generalização dos conceitos de vetor e matriz. É um arranjo multi-dimensional ou uma matriz de mais de duas dimensões. Em matemática, tensor possui outra definição. Porém, para nós, um tensor é simplesmente uma matriz multi-dimensional.

A forma específica de instalar depende do hardware do seu computador e da opção de instalação escolhida (se vai usar ou não GPU; do modelo do GPU; se vai instalar em ambiente virtual ou não; se vai usar Anaconda ou não, etc). Instale como você achar melhor.

Nota: Deixei no site abaixo as instruções de instalação, mas devem estar desatualizadas:

http://www.lps.usp.br/hae/software/instalacao_tensorflow.pdf

Keras em TensorFlow/PyTorch roda consideravelmente mais rápido em GPU do que CPU (algo como 3-10 vezes mais rápido). Por outro lado, dá muito mais trabalho instalá-los para usar GPU do que CPU.

Nota: A dificuldade para instalar TensorFlow para GPU é causado pela necessidade de instalar software de versões compatíveis: sistema Linux, Python, driver de nvidia, CUDA toolkit, cuDNN, Tensorflow, PyTorch. TensorFlow para GPU não funciona se instalar as últimas versões de todos os softwares.

[PSI3471 aula 10. Pular a partir daqui.]

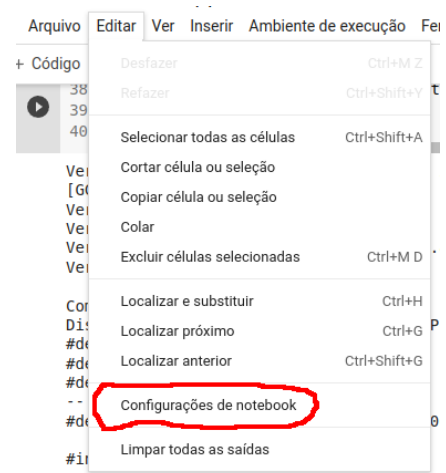
1.1 Google Colab

a) Uma outra forma de usar Keras/Tensorflow com GPU, sem precisar instalá-las no computador local, é através do Google Colab. Você pode entrar no Colab via browser (Chrome, Firefox, Brave, Chromium, etc.) com sua conta Google, dar copy-paste dos programas desta apostila e tudo executa corretamente (talvez com algumas pequenas adaptações). Inclusive, dá para usar GPU/TPU bom só editando as configurações em “notebook settings”. Você praticamente não precisa instalar nada no Google Colab – quase todas as bibliotecas estão instaladas de forma compatíveis e funcionando.

GPU (Graphics Processing Unit): Originalmente projetada para renderização gráfica (como em jogos), a GPU é uma unidade de processamento paralelo com milhares de núcleos pequenos e eficientes. GPUs são amplamente usadas para acelerar operações de matriz e vetor, que são fundamentais em cálculos de redes neurais. Pode ser usada tanto por TensorFlow como PyTorch.

TPU (Tensor Processing Unit): Desenvolvida pelo Google, a TPU é uma unidade de processamento especializada para operações de tensores (matrizes multidimensionais), com foco em IA e deep learning. TPUs são otimizadas para treinamento e inferência de modelos de aprendizado profundo, especialmente em grandes escalas. Possui integração nativa com TensorFlow, mas é mais difícil de usar em PyTorch.

Para usar GPU ou TPU dentro de Google Colab, você deve editar “editar → configurações de notebook”.



The image shows a screenshot of the Google Colab interface. On the left, there is a text box containing the instruction: "Para usar GPU ou TPU dentro de Google Colab, você deve editar “editar → configurações de notebook”." On the right, the 'Edit' menu is open, displaying various options. The option 'Configurações de notebook' is highlighted with a red circle. The menu also includes options like 'Desfazer', 'Refazer', 'Selecionar todas as células', 'Cortar célula ou seleção', 'Copiar célula ou seleção', 'Colar', 'Excluir células selecionadas', 'Localizar e substituir', 'Localizar próximo', 'Localizar anterior', and 'Limpar todas as saídas'.

Há alguns outros serviços semelhantes a Google Colab, entre eles:

- Kaggle: <https://www.kaggle.com/> - Disponibilizava TPUs com mais memória que Colab (2022).
- Paperspace gradient: <https://www.paperspace.com/gradient> – Pagando, consegue usar GPUs melhores que Colab (2022).
- AWS

Após instalar Keras/TensorFlow no seu computador local, rode o programa abaixo usando o comando [~/deep/keras/versao]:

Linux\$ python3 versao3-local.py
Windows> python versao3-local.py

```
#versao3-local.py - para computador local
#Imprime versao de TensorFlow/Keras.
#Tambem imprime o modelo do GPU e se TensorFlow consegue usa-lo.
#Imprime versao da Cuda, CUDNN, etc.
import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
os.environ['TF_FORCE_GPU_ALLOW_GROWTH'] = 'true'
import tensorflow as tf
import tensorflow.keras as keras
import sys, cv2

print("Versao python3:",sys.version)
print("Versao de tensorflow:",tf.__version__)
print("Versao de OpenCV:",cv2.__version__)
print()

gpu=tf.test.gpu_device_name()
if gpu=="":
    print("Computador sem GPU.")
else:
    print("Computador com GPU:",tf.test.gpu_device_name())
    from tensorflow.python.client import device_lib
    devices=device_lib.list_local_devices()
    print("Dispositivos:",[x.physical_device_desc for x in devices if x.physical_device_desc!=""])
    os.system("cat /usr/include/cudnn_version.h | grep '#define CUDNN_MAJOR' -A 2")
    os.system("nvcc --version | grep release")
    os.system('nvidia-smi')
    print()

os.system('lsb_release -a | grep "Description"') #imprime qual é o sistema operacional
os.system('cat /proc/cpuinfo | grep -E "model name"') #especificações de CPU
os.system('cat /proc/meminfo | grep "Mem"') #especificações de RAM
print()

import torch;
print("Versao pytorch: ",torch.__version__);
print("GPU disponivel em pytorch: ",torch.cuda.is_available());
```

Programa versao3-local.py: Imprime versões das bibliotecas no computador local.

A saída obtida executando esse programa no meu computador está abaixo. Destaquei em cores as informações importantes: Python 3.8.10, TensorFlow 2.13.1, GPU RTX 2060 com 6GB, CUDNN 8.6.0, Cuda 12.4.

```
Versao python3: 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0]
Versao de tensorflow: 2.19.0
Versao de OpenCV: 4.11.0

Computador com GPU: /device:GPU:0
Dispositivos: ['device: 0, name: NVIDIA GeForce RTX 4070 Laptop GPU, pci bus id: 0000:01:00.0, compute capability: 8.9']
Tue Apr 7 08:39:11 2026

+-----+
| NVIDIA-SMI 535.288.01                  Driver Version: 535.288.01   CUDA Version: 12.2     |
+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0    NVIDIA GeForce RTX 4070 ...        Off | 00000000:01:00.0 On  |          N/A         |
| N/A   48C    P0              2W / 80W    | 180MiB / 8188MiB |    10%    Default   |
+-----+-----+
+-----+
| Processes:                               GPU Memory |
|  GPU   GI    CI        PID   Type   Process name                  Usage |
|=====+=====+
| 0     N/A   N/A     1679    G   /usr/lib/xorg/Xorg              45MiB |
| 0     N/A   N/A    14277    C   python3                       126MiB |
+-----+

Description:Linux Mint 22.3
model name : Intel(R) Core(TM) i9-14900HX
(...)
model name : Intel(R) Core(TM) i9-14900HX
MemTotal: 32556168 kB
MemFree: 21195112 kB
MemAvailable: 26028264 kB

Versao pytorch: 2.7.0+cu126
GPU disponivel em pytorch: True
```

No Google Colab, selecione uma máquina virtual com GPU (Editar → Configurações do Notebook → Acelerador de hardware GPU) e execute o programa abaixo [Colab Notebooks/algpi/versao3]:

```
#versao3-colab.py - para Colab
#Imprime versao de TensorFlow/Keras.
#Tambem imprime o modelo do GPU e se TensorFlow consegue usa-lo.
#Imprime versao da Cuda, CUDNN, etc.
import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
os.environ['TF_FORCE_GPU_ALLOW_GROWTH'] = 'true'
import tensorflow as tf
import tensorflow.keras as keras
import sys, cv2

print("Versao python3:",sys.version)
print("Versao de tensorflow:",tf.__version__)
print("Versao de OpenCV:",cv2.__version__)
print()

gpu=tf.test.gpu_device_name()
if gpu=="":
    print("Computador sem GPU.")
else:
    print("Computador com GPU:",tf.test.gpu_device_name())
    from tensorflow.python.client import device_lib
    devices=device_lib.list_local_devices()
    print("Dispositivos:",[x.physical_device_desc for x in devices if x.physical_device_desc!=""])
    !cat /usr/include/cudnn_version.h | grep '#define CUDNN_MAJOR' -A 2
    !nvcc --version | grep release
    !nvidia-smi
    print()

    !lsb_release -a | grep "Description" #imprime qual é o sistema operacional
    !cat /proc/cpuinfo | grep -E "model name" #especificações de CPU
    !cat /proc/meminfo | grep "Mem" #especificações de RAM
    print()

import torch;
print("Versao pytorch: ",torch.__version__);
print("GPU disponivel em pytorch: ",torch.cuda.is_available());
```

Programa versao3-colab.py: Imprime versões das bibliotecas no Google Colab.

https://colab.research.google.com/drive/1_PlnkwdMB-BwCQhLd0tmt61SC6e0bdb?usp=sharing



Rodando esse programa no Colab com GPU, a saída indica: Python 3.10.12, TensorFlow 2.15.0, GPU Tesla T4 com 16GB, CUDNN 8.9.6, Cuda 12.2.

```
Versao python3: 3.12.13 (main, Mar 4 2026, 09:23:07) [GCC 11.4.0]
Versao de tensorflow: 2.19.0
Versao de OpenCV: 4.13.0

Computador com GPU: /device:GPU:0
Dispositivos: ['device: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5']
#define CUDNN_MAJOR 9
#define CUDNN_MINOR 8
#define CUDNN_PATCHLEVEL 0
Cuda compilation tools, release 12.8, V12.8.93
Tue Apr 7 11:47:16 2026

+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07          CUDA Version: 13.0     |
+-----+-----+
| GPU   Name           Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0   Tesla T4               Off   | 00000000:00:04.0 Off  |             0        |
| N/A   55C    P0              26W / 70W | 105MiB / 15360MiB |      4%      Default  |
+-----+-----+

+-----+
| Processes:                        |
| GPU   GI    CI        PID   Type   Process name                  GPU Memory |
| ID     ID     ID              |                 |           Usage |
|=====+=====+
| 0   N/A   N/A         577    C    /usr/bin/python3              102MiB    |
+-----+

No LSB modules are available.
Description: Ubuntu 22.04.5 LTS
model name : Intel(R) Xeon(R) CPU @ 2.00GHz
model name : Intel(R) Xeon(R) CPU @ 2.00GHz
MemTotal: 13286948 kB
```

```

MemFree:      7932376 kB
MemAvailable: 11714052 kB

Precisao padrao para float: float32
Versao Keras: 3.13.2
Versao pytorch: 2.10.0+cu128
GPU disponivel em pytorch: True

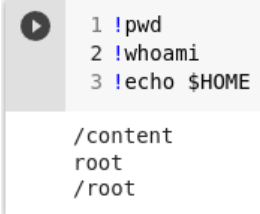
```

Há duas bibliotecas Keras distintas:

- A) Uma biblioteca Keras independente do TensorFlow (“import keras”); e
- B) Uma outra biblioteca Keras que vem incluída dentro do TensorFlow (“import tensorflow.keras as keras”).

Use sempre Keras incluída dentro do Tensorflow (opção B), pois algumas funções só existem nessa versão.

b) Associado à sua conta Google, você tem acesso a Google Drive e Colab. Os “notebooks” (programas Python) que você cria em Colab são salvos normalmente no diretório “Colab Notebooks” do seu Drive e não desaparecem quando você fecha a sessão Colab. Porém, todos os arquivos que você deixar no Colab (exceto os “notebooks”) desaparecem quando você fecha a seção. Se você instalou alguma biblioteca dentro do Colab, esta biblioteca também não estará mais disponível quando você iniciar uma nova seção. Explicarei adiante no item (f) como contornar isto.

c) Quando você entra no computador virtual do Google Colab, o seu diretório atual é “/content”, você é o usuário “root” e o seu diretório \$HOME é /root.	
--	--

d) Os comandos de bash (terminal de Linux) podem ser executados dentro do Colab em “subshell” prefixando o comando com “!”. Por exemplo, escrevendo “!ls” no Colab, irá obter o conteúdo do diretório atual. Se escrever “!pwd”, obterá o seu diretório default.

```

!ls #lista conteúdo do diretório atual (inicialmente “sample_data”)
!pwd #imprime qual é o diretório atual (normalmente “/content”)

```

Nota: É também possível usar o comando `os.system(“...”)` de Python em vez de “!”. O problema é que a saída no terminal deste comando fica invisível.

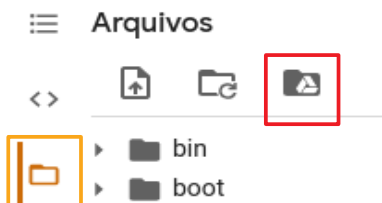
e) Para mudar o diretório atual “!cd” não funciona, pois esse comando é executado em subshell e a mudança de diretório atual é desfeita quando termina subshell. Para mudar o diretório permanentemente, deve usar “%cd” onde “%” prefixa “magic commands”:

```

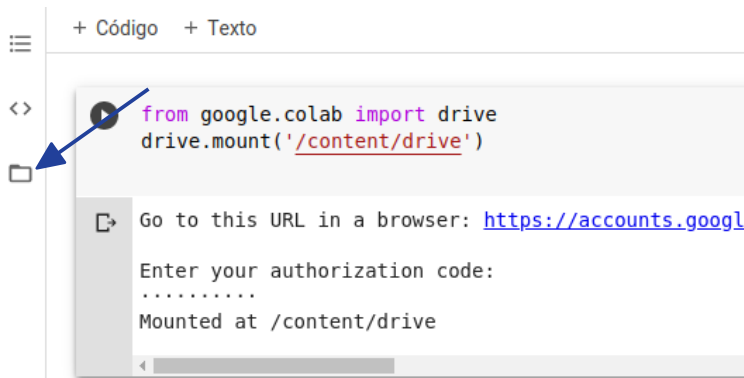
%cd diretorio [muda para diretorio]
%reset -f [apaga todas as variáveis e funções de Python]

```

f) Como já disse no item b, Colab não armazena permanentemente os arquivos dentro do seu computador virtual. Os arquivos são todos excluídos quando a seção Colab é fechada. Uma forma de armazenar arquivos permanentemente é você montar o seu Google Drive como um subdiretório do sistema de arquivos de Colab e deixar lá os seus arquivos. Isto pode ser feito rodando o código Python (na figura abaixo, direita). Se você clicar no botão laranja e depois no botão vermelho (na figura abaixo, esquerda), Colab cria essa célula.

	<pre> from google.colab import drive drive.mount('/content/drive') </pre>
---	---

Depois que o diretório estiver montado, você pode visualizar a sua estrutura de diretórios clicando em:



O seu Google Drive ficará montado no diretório:
"/content/drive/MyDrive"

Nota: Apesar do seu Google Drive parecer um diretório local Colab (/content/drive/MyDrive/) o seu Google Drive pode estar fisicamente bem distante do computador Colab. Assim, a transferência de arquivos entre Google Drive e Colab pode ser lenta, principalmente a transferência de vários arquivos pequenos. É muito mais rápido transferir um arquivo compactado (.zip) grande entre Google Drive e Colab do que transferir vários arquivos pequenos. Outra forma de acelerar a transferência de arquivos é trabalhar no HD local (/content) e transferir os arquivos para Google Drive no final da execução.

Nota: "cv2.imshow" do OpenCV não funciona dentro do Colab. No seu lugar, deve usar "plt.imshow" do pyplot.

```
from matplotlib import pyplot as plt
a=plt.imread("lenna.jpg")
plt.imshow(a); plt.axis("off"); plt.show()
```

Outra opção é usar cv2_imshow:

```
from google.colab.patches import cv2_imshow
cv2_imshow(img)
```

Outras dicas sobre Google Colab estão em Anexo B.

1.3 NumPy e Tensores

Para poder trabalhar com aprendizado de máquina em Python, é necessário manipular tensores (matrizes multidimensionais). Listo abaixo os comandos que considero os mais essenciais, para quem está iniciando.

1) *NumPy*. Provavelmente, todas as bibliotecas de aprendizado de máquina em Python (OpenCV, Keras, Tensorflow, PyTorch, etc) ou trabalham diretamente com tensores de NumPy ou possuem funções para importar/exportar tensores de/para NumPy. Assim, NumPy é uma espécie de “biblioteca universal” em Python para trabalhar com tensores.

2) *Descobrir o tipo de variável*. Como em Python o tipo de variável é dinâmico (pode ficar mudando de tipo ao longo do programa), é importante saber descobrir o tipo de variável **v** num certo ponto do programa. Para isso, use **“type(v)”**. O tipo de um tensor NumPy é **“numpy.ndarray”**.

3) *Tamanho do tensor*. O número de dimensões do tensor pode ser determinado com **“len(v.shape)”**. As dimensões de um tensor NumPy podem ser determinadas com **“v.shape”**.

4) *Tipo do tensor*. O tipo de elemento de um tensor NumPy pode ser determinado com **“v.dtype”**.

O exemplo abaixo cria um tensor NumPy 3×4 tipo “unsigned int 8” ou “uchar”. Depois, imprime o tipo de variável (class ‘numpy.ndarray’), o número de dimensões (2), o tamanho (3,4) e o tipo de elemento (uint8).

Exemplo:

Programa	Saída
<pre>import numpy as np v=np.empty((3,4),dtype=np.uint8) print(type(v)) print(len(v.shape)) print(v.shape) print(v.dtype)</pre>	<pre><class 'numpy.ndarray'> 2 (3, 4) uint8</pre>

As outras funções de NumPy ou de outras bibliotecas se encontram nos respectivos manuais e tutoriais.

[PSI3471 aula 10. Pular até aqui.]

Recordação: Aprendizado supervisionado:

Vamos recordar a nomenclatura adotada. No aprendizado supervisionado, um “professor” fornece amostras ou exemplos de treinamento de entrada/saída (ax, ay) . O “algoritmo de aprendizado” M classifica uma nova instância de entrada qx baseado nos exemplos fornecidos pelo professor, gerando a classificação qp . Para cada instância de entrada qx existe uma classificação correta qy . Se a classificação do algoritmo $qp=M(qx)$ for igual à classificação verdadeira qy então o algoritmo acertou a classificação.

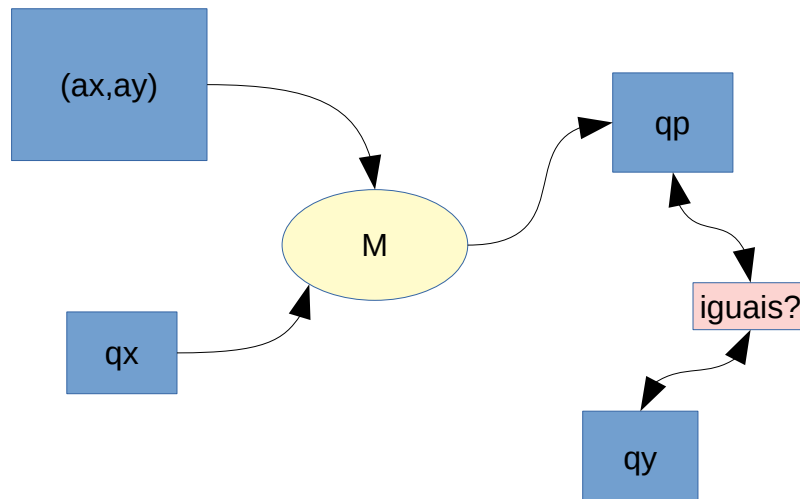


Figura: Nomenclatura utilizada no aprendizado supervisionado.

Nota: Muitos outros autores chamam:

ax	$train_x$
ay	$train_y$
qx	$test_x$
qy	$test_y$
qp	$test_y$

2. Regressão em Keras (versão nova)

Após instalar Python3/TensorFlow/Keras (ou usando Colab), vamos criar uma rede neural simples, rasa e com camadas densas (também chamado de *fully-connected* ou *inner-product*). Vamos criar uma rede neural com 2 entradas e 1 saída. Queremos que a rede gere saída 1 para a entrada (1,0); e gere saída 0 para as entradas (0,1), (0,0) e (1,1). A figura F1 mostra o comportamento desejado da rede.

```
AX = np.matrix('1 0; 0 1; 0 0; 1 1', dtype='float32')
AY = np.matrix('1; 0; 0; 0', dtype='float32')
```

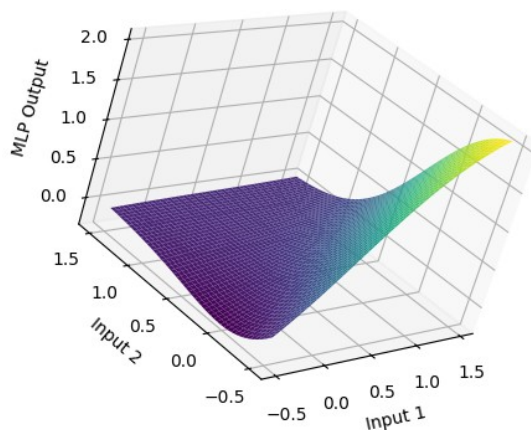


Figura F1: A saída da rede neural do programa 1a em função das duas entradas.

A figura 1 mostra a estrutura da rede que possui duas entradas (i1 e i2, bolinhas azuis), uma saída (bolinha cinza) e três neurônios (h1, h2 e o1) organizados em duas camadas densas (verde e amarela). Esses três neurônios possuem ao todo 9 parâmetros (6 pesos e 3 vieses) que devem ser ajustados para a rede se comportar da forma desejada.

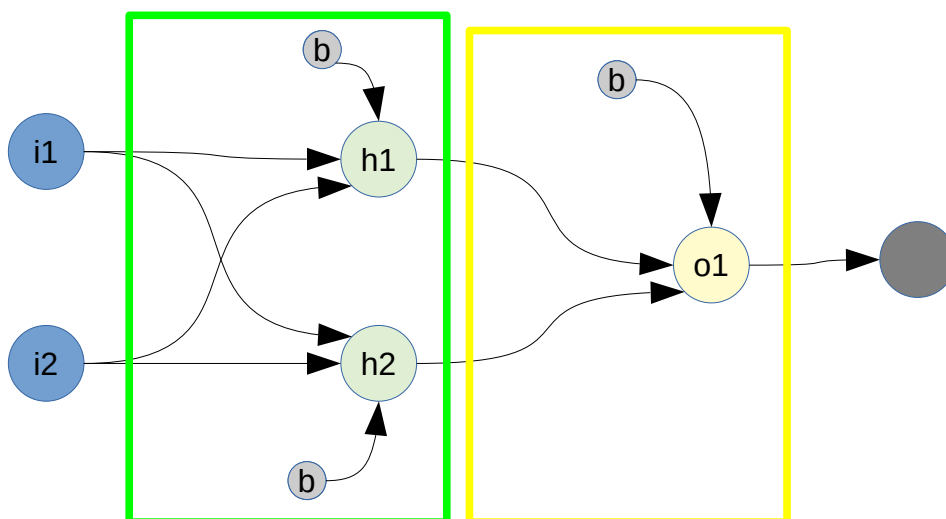


Figura 1: Estrutura da rede neural do programa regression.py

A implementação desta rede em Keras/Tensorflow está no programa 1a.

As linhas 2-9 importam os módulos a serem usados no programa. A linha 2 indica que não queremos que TensorFlow mostre informações excessivas. As linhas 11-14 constroem o modelo da rede sequencial da figura 1. Uma rede sequencial é formada por uma sequência linear de camadas (sem desvios nem recorrências).

Nota: Utilize sempre o módulo “tensorflow.keras” e não o módulo “keras”, pois o primeiro é a implementação de Keras específica para Tensorflow e acrescenta várias facilidades para Tensorflow.

```
1  #~/deep/algpi/densa/regression2/regression2.py - 2026
2  import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
3  import tensorflow.keras as keras
4  from tensorflow.keras.models import Sequential
5  from tensorflow.keras.layers import Dense, Activation
6  from tensorflow.keras import optimizers
7  import numpy as np
8  import matplotlib.pyplot as plt
9  from mpl_toolkits.mplot3d import Axes3D
10
11 #Define modelo de rede
12 model = Sequential()
13 model.add(Dense(2, activation='sigmoid', input_dim=2))
14 model.add(Dense(1, activation='linear'))
15
16 sgd=optimizers.SGD(learning_rate=0.2)
17 model.compile(optimizer=sgd, loss='mse')
18
19 AX = np.matrix('1 0; 0 1; 0 0; 1 1', dtype='float32')
20 AY = np.matrix('1; 0 ; 0 ; 0', dtype='float32')
21 print("AX"); print(AX)
22 print("AY"); print(AY)
23
24 # Print the initial parameters
25 print("\nInitial MLP Parameters:")
26 for layer in model.layers:
27     weights = layer.get_weights()
28     if weights: # Check if the layer has weights
29         print(f"Layer: {layer.name}")
30         print(f"  Weights: \n{weights[0]}")
31         print(f"  Biases: \n{weights[1]}")
32
33 # As opcoes sao usar batch_size=2 ou 1
34 model.fit(AX, AY, epochs=1000, batch_size=2, verbose=2)
35
36 # Print the trained parameters
37 print("\nTrained MLP Parameters:")
38 for layer in model.layers:
39     weights = layer.get_weights()
40     if weights: # Check if the layer has weights
41         print(f"Layer: {layer.name}")
42         print(f"  Weights: \n{weights[0]}")
43         print(f"  Biases: \n{weights[1]}")
44
45 QX = np.matrix('1 0; 0 1; 0 0; 1 1', dtype='float32')
46 print("QX"); print(QX)
47 QP=model.predict(QX, verbose=2)
48 print("QP"); print(QP)
49
50 # Generate data for 3D plot
51 x1 = np.linspace(-0.5, 1.5, 50)
52 x2 = np.linspace(-0.5, 1.5, 50)
53 x1_grid, x2_grid = np.meshgrid(x1, x2)
54 QX_3d = np.hstack((x1_grid.reshape(-1, 1), x2_grid.reshape(-1, 1)))
55 QP_3d = model.predict(QX_3d)
56
57 # Plot the 3D surface
58 fig = plt.figure()
59 ax = fig.add_subplot(111, projection='3d')
60 ax.plot_surface(x1_grid, x2_grid, QP_3d.reshape(x1_grid.shape), cmap='viridis')
61 ax.set_xlabel('Input 1')
62 ax.set_ylabel('Input 2')
63 ax.set_zlabel('MLP Output')
64 plt.show()
```

Programa 1a: Regression.py (com backend TensorFlow)

https://colab.research.google.com/drive/12gGDv3QAau0R0HNc_qDkGc7ZJlctpMSI
[~/deep/algpi/densa/regression2/regression2.py] [Colab Notebooks/algpi/densa/regression2.ipynb]

Cada neurônio da rede (figura 1) recebe as duas entradas i_1 e i_2 (isto é, dois números em ponto flutuante), multiplica-as pelos respectivos pesos w_1 e w_2 , soma o viés (bias) b e aplica uma função de ativação não-linear ao resultado (no exemplo, a função sigmoide), gerando a saída a (um número em ponto flutuante). A figura 2 ilustra o funcionamento de um neurônio da rede.

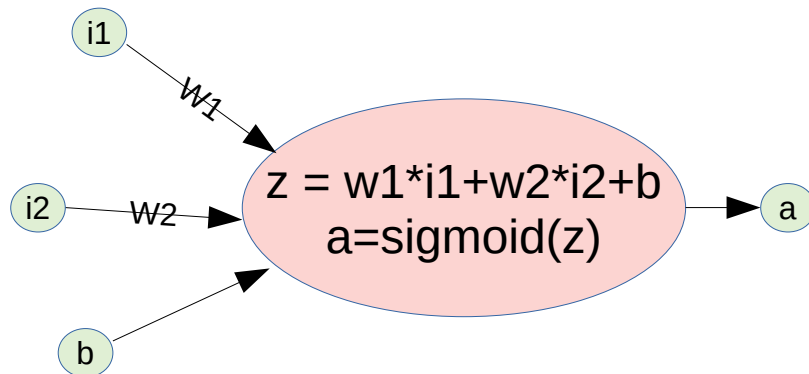


Figura 2: Funcionamento de um neurônio.

A função “sigmoide” ou “logistic” é definida como (figura 3):

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

A entrada x de sigmoide é qualquer número real e a saída fica limitada entre zero e um.

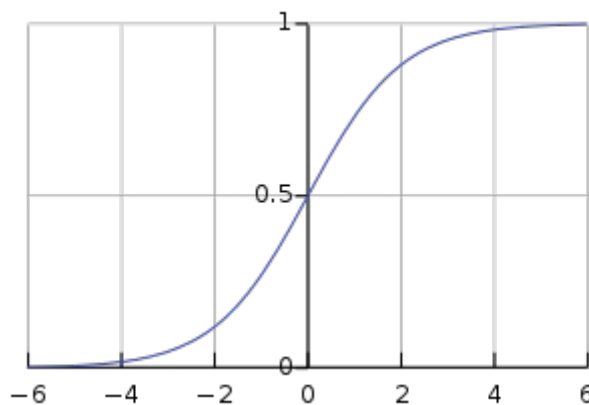


Figura 3: Função sigmoide ou logistic.

No programa, a linha 13 especifica que queremos usar a função de ativação sigmoide na primeira camada. A linha 14 especifica que não queremos usar função de ativação na segunda camada (a saída da função de ativação “linear” é igual à entrada).

Na figura 1, os 3 neurônios estão organizados em 2 camadas: verde e amarela. A camada verde (h_1 e h_2) é a camada escondida (linha 13 do programa 1) que recebe os dados de entrada. A camada amarela (o_1) é a camada de saída (linha 14 do programa).

Uma rede neural M possui uma função custo (ou de erro ou de perda) C que mede o quanto a saída $qp=M(qx)$, para a entrada qx , é diferente da saída desejada qy : $C(q) = C(M(qx), qy)$. No nosso programa, vamos usar o erro quadrático médio (“mean squared error” ou “mse”, linha 17).

Retro-propagação (backpropagation) é o processo que ajusta, aos poucos, os parâmetros da rede para diminuir a função custo (diferença entre a saída da rede e a saída desejada), alterando “um pouco” os valores dos pesos e dos bias. Para isso, devemos calcular as derivadas parciais da função custo C em relação a cada peso e cada viés:

$$\frac{\partial C}{\partial w_k} \text{ e } \frac{\partial C}{\partial b_l}$$

Nota: A apostila autodiff explica como as derivadas parciais são calculadas.

A função custo é calculada nos exemplos de treinamento organizados em lotes (mini-batches). O nosso programa usa lotes com 2 exemplos de treino por vez (`batch_size=2`, linha 25).

Cada derivada parcial indica para onde se deve mover para aumentar a função custo. Como queremos diminuir a função custo, movemos “um pouco” cada peso w e cada viés b no sentido contrário à sua derivada parcial (figura 4). Para especificar “um pouco”, utiliza-se a taxa de aprendizado (learning rate) η :

$$w_k \rightarrow w'_k = w_k - \eta \frac{\partial C}{\partial w_k}$$

$$b_l \rightarrow b'_l = b_l - \eta \frac{\partial C}{\partial b_l}$$

No programa 1, a taxa de aprendizado (`learning_rate`) está especificada na linha 16 como sendo 0,2. Normalmente, utiliza-se uma taxa de aprendizado bem menor, como 0,001. Porém, neste exemplo simples, é possível usar taxa de aprendizado grande.

Existem três formas de fazer descida do gradiente.

1. *Batch gradient descent* ou simplesmente *gradient descent* utiliza todas os m dados de treino para calcular a função custo médio C . Depois, efetua-se retro-propagação.
2. *Stochastic gradient descent* utiliza um único dado de treino para calcular a função de custo C e atualiza os parâmetros.
3. *Mini-batch gradient descent*: Na descida de gradiente em mini-lotes, a função custo é calculada em subconjuntos de dados de treino (lotes).

Em Keras, “stochastic gradient descent” (*sgd* na linha 17) pode significar qualquer uma das três estratégias acima, dependendo do parâmetro `batch_size` escolhido (linha 25).

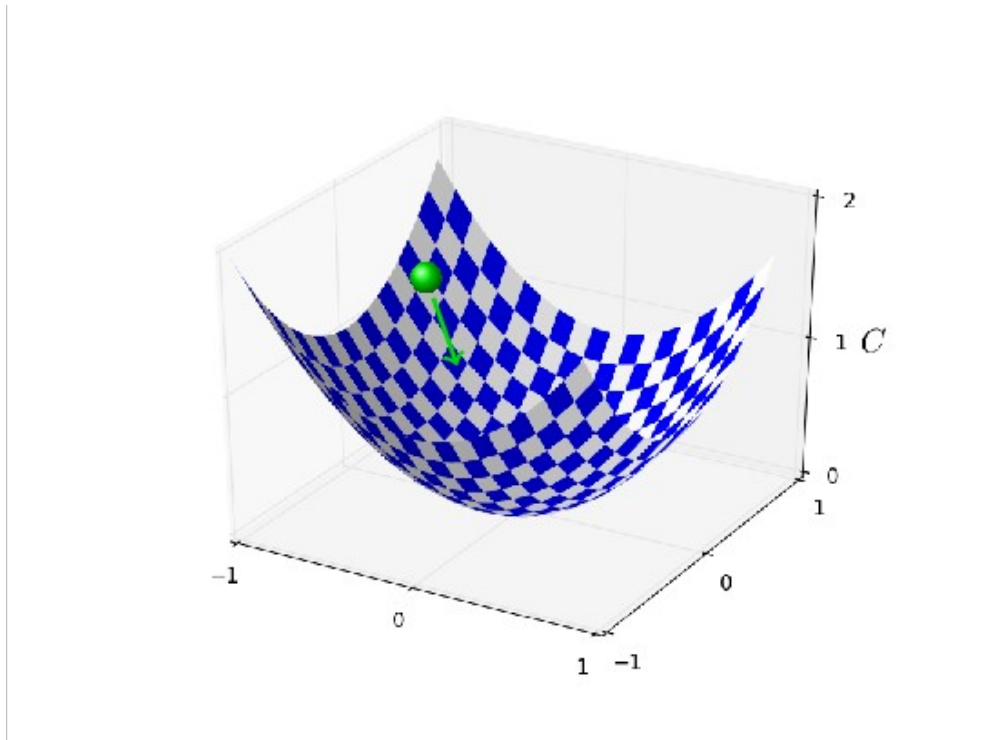


Figura 4: Retro-propagação muda os valores dos pesos e viés para diminuir a função custo (descida do gradiente, figura extraída de [Nielsen]).

A função `model.fit` (linha 25 do programa 1) executa a retro-propagação da rede neural, com 1000 épocas (epochs). Cada época corresponde a executar a descida de gradiente para todos os lotes, até que todas as amostras de treino tenham sido usadas. No nosso caso, cada época vai executar duas descidas de gradiente (pois o número de amostras é 4 e tamanho do mini-batch é 2).

Nas linhas 36-39, aplicamos a rede neural treinada nos dados de teste.

Executando o programa, obtemos:

```
AX
[[1. 0.]
 [0. 1.]
 [0. 0.]
 [1. 1.]]
AY
[[1.]
 [0.]
 [0.]
 [0.]]

Initial MLP Parameters:
Layer: dense_2
Weights:
[[-1.0161297  0.3917539]
 [ 0.9741026 -0.4071985]]
Biases:
[0. 0.]
Layer: dense_3
Weights:
[[-1.2381054 ]
 [ 0.15698421]]
Biases:
[0.]

Epoch      2/3000 2/2 - 0s - 13ms/step - loss: 0.0941
Epoch 1500/3000 2/2 - 0s - 12ms/step - loss: 3.2558e-04
Epoch 3000/3000 2/2 - 0s - 13ms/step - loss: 5.1711e-07

Trained MLP Parameters:
Layer: dense_2
Weights:
[[-2.6587064 -0.30026805]
 [ 3.2054014  0.62948674]]
Biases:
[ 2.341416 -0.23308145]
Layer: dense_3
Weights:
[[-2.1401575 ]
 [ 0.93510234]]
Biases:
[1.5479347]
QX
[[1. 0.]
 [0. 1.]
 [0. 0.]
 [1. 1.]]
1/1 - 0s - 90ms/step
QP
[[ 0.9919499 ]
 [-0.02488267]
 [ 0.00888479]
 [ 0.01066697]]
```

Veja como a função custo foi diminuindo com o aumento das épocas. O programa imprime também os pesos e vieses finais.

As saídas QP obtidas estão de acordo com o treino. A cada execução, a saída será um pouco diferente, pois o programa inicializa os pesos e os vieses aleatoriamente (veja Anexo A.1).

[PSI3471 aulas 9/10lição de casa #2/2. Vale 3.] Modifique o programa 1a para obter:

```
AX = np.matrix('0.5 0.5; 0.0 0.0; 0.0 1.0; 1.0 0.0; 1.0 1.0;
               0.5 0.0; 0.0 0.5; 0.5 1.0; 1.0 0.5', dtype='float32')
AY = np.matrix('1.0; 0.0; 0.0; 0.0; 0.0;
               0.0; 0.0; 0.0; 0.0', dtype='float32')
```

Isto é, a saída da rede deve ser 1 para entrada (0.5, 0.5); e 0 para as entradas (0, 0); (0, 1); (1, 0); (1, 1); (0.5, 0); (0, 0.5); (0.5, 1); (1, 0.5).

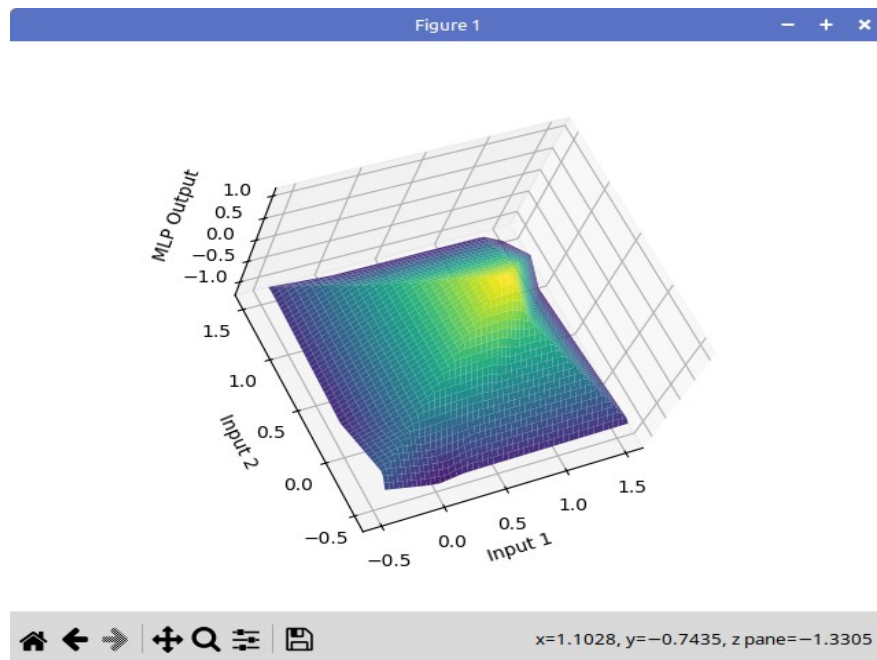


Figura F2: Saída desejada.

- a) É possível que a rede neural do programa 1a gere a saída desejada sem alterar a estrutura (usando 3 neurônios)?
- b) Modifique a estrutura do programa 1a para gerar a saída desejada. Qual é o menor número de neurônios necessário para gerar a saída desejada?

Solução em `~/deep/algpi/densa/regression2/regression5.py`

[PSI3471. Pular a partir daqui]

3. Classificação ABC (adulto, bebê ou criança) em Keras

Vamos resolver em Keras o problema de classificar indivíduos em “adulto, bebê ou criança” a partir do seu peso em kg, usando rede neural artificial.

Nota: Numa aula passada, resolvemos este problema usando vizinho mais próximo e árvore de decisão.

Figura 5 mostra uma possível estrutura da rede para resolver este problema. Cada “flecha” da rede possui um peso associado e cada neurônio da rede possui um viés associado. Para simplificar, vamos chamar de “parâmetros da rede” o conjunto formado pelos pesos e vieses. O problema é encontrar os parâmetros que, dado o peso de um indivíduo em *kg*, o classifiquem corretamente em A, B ou C.

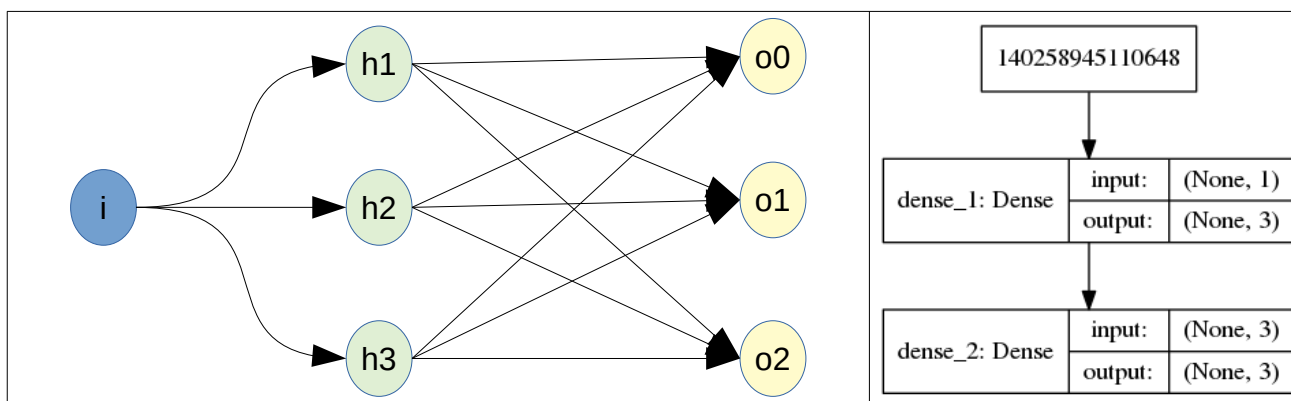


Figura 5: Estrutura da rede neural para resolver problema “Adulto”, “Bebê” e “Criança”.

Para usar rede neural, vamos converter os rótulos A, B e C em vetores de categorias “one-hot encoding” (1, 0, 0), (0, 1, 0) e (0, 0, 1), pois a rede da figura 5 possui três saídas (tabelas 1 e 2). Precisamos encontrar os parâmetros que fazem a classificação desejada. Para isso, o programa inicializa aleatoriamente os parâmetros (veja Anexo A.1 para maiores detalhes). Para treinar a rede, o programa efetua a retro-propagação.

Vou descrever intuitivamente a *retro-propagação* usando mini-batch de 1 elemento. O programa apresenta à rede um exemplo de treinamento (por exemplo “4”) e pega a saída fornecida pela rede. A saída desejada é o vetor (0, 1, 0) significando “Bebê”. Então, o programa aumenta ou diminui um pouco cada parâmetro para que a saída obtida fique um pouco mais próxima da desejada.

Tabela 1: Amostras de treinamento (*ax*, *ay*) e vetor de categoria *ay2*.

<i>ax</i> (características, entradas)	<i>ay</i> (rótulos, saídas)	<i>ay2</i> (one-hot-encoding)
4	B	0 1 0
15	C	0 0 1
65	A	1 0 0
5	B	0 1 0
18	C	0 0 1
70	A	1 0 0

Tabela 2: Instâncias de teste a classificar (*qx*), classificação verdadeira (*qy*) e vetor de categoria verdadeira (*qy2*).

<i>qx</i> (instância de teste)	<i>qy</i> (classificação verdadeira)	<i>qy2</i> (one-hot-encoding)
16	C	0 0 1
3	B	0 1 0
75	A	1 0 0

A implementação em Keras está no programa 2.

```
1 #abc1.py - 2024
2 import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
3 import tensorflow as tf
4 import tensorflow.keras as keras
5 from tensorflow.keras.models import Sequential
6 from tensorflow.keras.layers import Dense, Activation
7 from tensorflow.keras import optimizers
8 import numpy as np
9 import sys
10
11 model = Sequential(); model.add(Input(shape=(1,)))
12 model.add(Dense(3, activation='sigmoid'))
13 model.add(Dense(3, activation='sigmoid'))
14 sgd=optimizers.SGD(learning_rate=10.0);
15 model.compile(optimizer=sgd, loss='mse', metrics=['accuracy'])
16
17 ax = np.matrix('4;      15;      65;      5;      18;      70      ',dtype="float32")
18 ax=2*(ax/100-0.5) #-1 a +1
19 ay = np.matrix('0 1 0; 0 0 1; 1 0 0; 0 1 0; 0 0 1; 1 0 0',dtype="float32")
20 model.fit(ax, ay, epochs=200, batch_size=2, verbose=2)
21
22 qx = np.matrix('16;      3;      75      ',dtype="float32")
23 qx=2*(qx/100-0.5) #-1 a +1
24 qy = np.matrix('0 0 1; 0 1 0; 1 0 0',dtype="float32")
25 teste = model.evaluate(qx,qy)
26 print("Custo e acuracidade de teste:",teste)
27
28 qp2=model.predict(qx)
29 print("Classificacao de teste:\n",qp2)
30 qp = qp2.argmax(axis=1)
31 print("Rotulo de saída:\n",qp)
32
33 from tensorflow.keras.utils import plot_model
34 plot_model(model, to_file='abc1.png', show_shapes=True)
35 model.summary()
```

Programa 2: Resolução do problema ABC em Keras.

https://colab.research.google.com/drive/1pNUbinzKFvQ_mAnB19RsjqB9GVpbn4eA?usp=sharing

Nota: Não pode salvar este programa com nome “abc.py”, pois parece que “abc” é uma palavra reservada em Python.

As linhas 11-15 especificam o modelo de rede, o otimizador “stochastic gradient descent” (sgd) com “learning rate” 10, e compila a rede usando “mean squared error” (mse) como medida de erro.

As linhas 17-19 especificam as entradas *ax* e saídas *ay* de treinamento. Estou normalizando o intervalo da entrada de 0 a 100 (kg) para -1 a +1. Experimente rodar o programa sem esta normalização – no meu teste, a rede não convergiu e a taxa de acerto final foi de 33%, equivalendo a “chute”.

Apesar de backpropagation conseguir ajustar os parâmetros para variáveis de entradas com distribuições arbitrárias, a rede converge mais facilmente quando as variáveis de entrada estão no intervalo “semelhante” a uma distribuição normal de média zero e desvio um (por exemplo, números no intervalo [-1, +1], [-0.5, +0.5], etc).

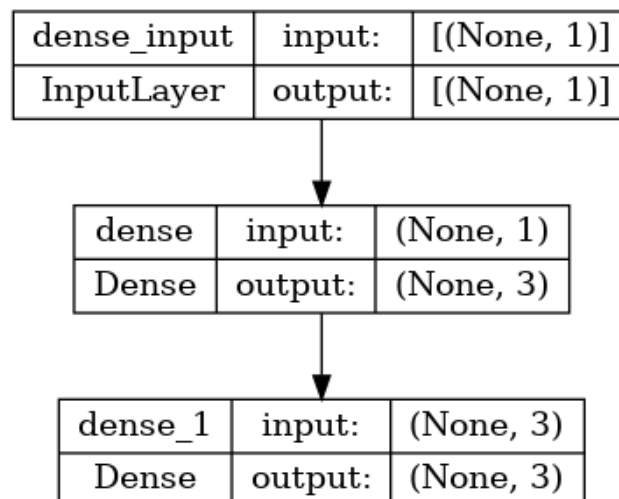
A linha 20 efetua retro-propagação em 200 épocas, usando mini-batches de tamanho 2.

As linhas 22-24 especificam as entradas *qx* e saídas *qy* de teste. A linha 25 calcula o custo e acuracidade da rede nos dados de teste.

Nas linhas 28-31, o programa efetua as inferências de *qx* como vetores one-hot-encoding *qp2* e os converte em rótulos *qp* calculando *argmax*. Os rótulos de saída [2 1 0] estão de acordo com o que deveria dar [C B A]. As linhas 33-35 imprimem o modelo de rede da figura 5. A saída obtida é:

```
python3 abc1.py
Using TensorFlow backend.
Epoch 1/200 - 1s - loss: 0.3036 - accuracy: 0.1667 - 789ms/epoch
Epoch 100/200 - 0s - loss: 0.0740 - accuracy: 0.8333 - 4ms/epoch
Epoch 200/200 - 0s - loss: 0.0040 - accuracy: 1.0000 - 5ms/epoch
Custo e acuracidade de teste: [0.0023460739757865667, 1.0]
Classificacao de teste:
[[4.6338744e-02 4.1173872e-02 9.0481442e-01]
 [3.3631730e-03 9.6565908e-01 6.3102752e-02]
 [9.6162802e-01 1.8744668e-05 3.9583176e-02]]
Rotulo de saida:
[2 1 0]
Model: "sequential"

Layer (type)                 Output Shape                 Param #
=====
dense (Dense)                 (None, 3)                   6
dense_1 (Dense)               (None, 3)                   12
=====
Total params: 18
Trainable params: 18
Non-trainable params: 0
```



A rede apresenta acuracidade de 100%!

O número total de parâmetros é 18, pois a rede possui 12 pesos (as flechas da figura 5) e 6 vieses (os neurônios da figura 5).

Exercício: Modifique o programa `abc1.py` para que aceite dois atributos na entrada: peso em kg e cor da pele (0=escuro, 100=claro). Treine a rede com os dados abaixo:

ax		ay
peso	cor da pele	
4	6	B
15	8	C
65	70	A
5	90	B
18	70	C
70	10	A

Faça teste com os dados abaixo:

Instância a processar ou a classificar (qx):		Classificação ideal ou verdadeira desconhecida (qy):	Classificação pelo aprendiz (qp):
peso	cor da pele		
20	6	C	
3	50	B	
75	55	A	
5	80	B	
25	10	C	
65	20	A	

A rede neural artificial conseguiu identificar “cor da pele” como ruído?

Nota: A resposta esperada é que a rede neural consiga classificar corretamente todos os indivíduos mesmo com o atributo “cor da pele”. Se o seu exercício classificar incorretamente, tem algum erro no seu programa. Apesar deste exercício ser extremamente simples, há alunos que cometem diferentes tipos de erros.

[Solução privada em `~/deep/algpi/densa/abc1/pele1.py`]

Exercício:

Nos sites:

<https://archive.ics.uci.edu/ml/datasets/banknote+authentication> OU

<https://machinelearningmastery.com/standard-machine-learning-datasets> item (5)

há o arquivo `data_banknote_authentication.txt` com 4 atributos extraídos de notas de banco verdadeiras (classe 0) e falsas (classe 1). Como o problema original é fácil demais, para dificultar a tarefa de classificação, acrescentei mais 3 atributos que são números aleatório entre -30 e +30.

Peguei as linhas pares desse arquivo (começando da linha 0) para treino e as ímpares para teste, gerando os arquivos `ax.txt`, `ay.txt`, `qx.txt` e `qy.txt`, conforme a convenção adotada nesta apostila. Esses arquivos estão em:

<http://www.lps.usp.br/hae/apostila/noisynote.zip>

Crie uma rede neural artificial em Keras que classifica as notas de teste (`qx.txt`) em verdadeiras (classe 0) e falsas (classe 1). Procure obter a menor taxa de erro possível Qual foi a taxa de erro de teste obtida?

Nota: Obtive erro de teste de 0,0% a 0,2% (esta taxa muda a cada execução).

Ajuda 1: O seguinte programa baixa o BD no seu diretório atual e o descompacta:

```
url='http://www.lps.usp.br/hae/apostila/noisynote.zip'
import os; nomeArq=os.path.split(url)[1]
if not os.path.exists(nomeArq):
    print("Baixando o arquivo",nomeArq,"para diretorio default",os.getcwd())
    os.system("wget -nc -U 'Firefox/50.0' "+url)
else:
    print("O arquivo",nomeArq,"ja existe no diretorio default",os.getcwd())
print("Descompactando arquivos novos de",nomeArq)
os.system("unzip -u "+nomeArq)
```

Ajuda 2: O seguinte código lê as matrizes `ax`, `ay`, `qx` e `qy`:

```
import numpy as np
def le(nomearq):
    with open(nomearq,"r") as f:
        linhas=f.readlines()
        linha0=linhas[0].split()
        nl=int(linha0[0]); nc=int(linha0[1])
        a=np.empty((nl,nc),dtype=np.float32)
        for l in range(nl):
            linha=linhas[l+1].split()
            for c in range(nc):
                a[l,c]=np.float32(linha[c])
    return a

ax = le("ax.txt"); ay = le("ay.txt")
qx = le("qx.txt"); qy = le("qy.txt")
```

[Solução acesso restrito: https://colab.research.google.com/drive/1FPHYI9C569EJmPq_2BmgkfjJ0K1BLEBR?usp=sharing]

[PSI3471. Pular até aqui.]

4. Leitura de MNIST em Keras

Vamos denotar como $ay2$, $qy2$ e $qp2$ os vetores “one-hot encoding” de ay , qy e qp respectivamente. Por exemplo, se a categoria $AY=5$, o vetor one-hot-encoding correspondente é um vetor de 10 elementos (número de categorias) completamente preenchido com zeros exceto na posição índice 5 (6º elemento) que será preenchido com um.

$AY[0]: 5$

$AY2[0]: [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]$

O banco de dados MNIST consiste de 60000 imagens de treino (AX) e 10000 imagens de teste (QX) 28×28 de dígitos manuscritos, juntamente com rótulos de 0 a 9 (AY e QY). O problema é treinar um algoritmo de aprendizado com (AX, AY) e fazer inferência com QX, obtendo QP. Quanto mais QP e QY forem iguais (maior taxa de acerto), melhor é o algoritmo.

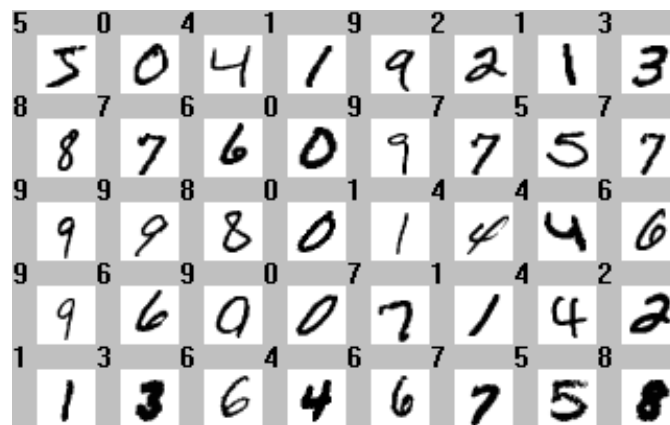


Figura 7: Alguns dígitos da MNIST, com as cores invertidas (as imagens originais possuem dígitos brancos sobre fundo preto). MNIST consiste de 60000 imagens de treino e 10000 imagens de teste 28×28.

Vamos classificar os dígitos do MNIST usando rede neural densa (não-convolucional) construído em Keras. Para isso, precisamos saber ler o banco de dados MNIST em Python.

```
1 # le_mnist.py 2024
2 import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'; import sys
3 import tensorflow.keras as keras
4 from tensorflow.keras.datasets import mnist
5 from matplotlib import pyplot as plt
6
7 (AX, AY), (QX, QY) = mnist.load_data()
8 print("AX:", AX.shape, AX.dtype)
9 print("AY:", AY.shape, AY.dtype)
10 print("QX:", QX.shape, QX.dtype)
11 print("QY:", QY.shape, QY.dtype)
12
13 AX=255-AX; QX=255-QX
14 plt.imshow(AX[0], cmap="gray", interpolation="nearest")
15 plt.show()
16
17 nclasses = 10
18 AY2 = keras.utils.to_categorical(AY, nclasses)
19 QY2 = keras.utils.to_categorical(QY, nclasses)
20 print("AY2:", AY2.shape, AY2.dtype)
21 print("QY2:", QY2.shape, QY2.dtype)
22 print("AY[0]:", AY[0])
23 print("AY2[0]:", AY2[0])
```

Programa 3: Leitura de MNIST em Python/Keras.

https://colab.research.google.com/drive/1nf7foxy9N_ZZJYRS1C4LiPlDuZdfhtbr?usp=sharing [~/deep/algpi/densa/mnist/le_mnist.py]

O programa 3 lê MNIST (linha 7), imprime os formatos e os tipos das matrizes lidas (linhas 8-11):

```
AX: (60000, 28, 28) uint8
AY: (60000,) uint8
QX: (10000, 28, 28) uint8
QY: (10000,) uint8
```

Isto significa que AX é um tensor com 60.000 imagens 28×28 em níveis de cinza. AY é um vetor de uint8 com 60.000 rótulos.

Nota: a vírgula em (60000,) indica que AX.shape é um vetor com apenas um elemento: 60000. Sem vírgula, (60000) indicaria o número 60000 e não um vetor com apenas um elemento 60000.

Depois, inverte preto com branco (linha 13) e mostra na tela a primeira imagem de treino (linhas 14-15, figura 8).

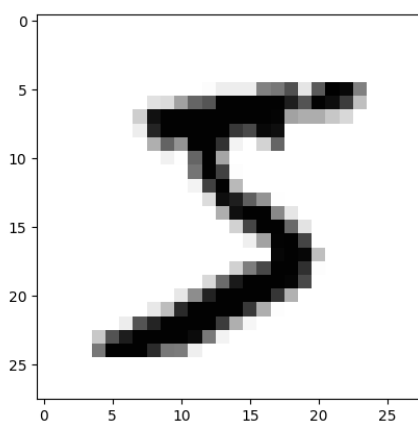


Figura 8: Imagem AX[0], primeira imagem de treino de MNIST, lida em Python.

Depois disso, o método *to_categorical* converte os rótulos AY e QY (números entre 0 e 9) em saídas categóricas “one-hot encoding” AY2 e QY2 (linhas 17-22), para poder alimentar uma rede neural com 10 neurônios de saída. As saídas impressas para compreender o que está acontecendo (linhas 20-22) são:

```
AY2: (60000, 10) float32
QY2: (10000, 10) float32
AY[0]: 5
AY2[0]: [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

Significando que a categoria AY[0] da primeira imagem de treino é “5”. Este rótulo foi convertido em one-hot-encoding com zero em todas as posições exceto na posição de índice 5 (6ª saída) que tem valor 1. Também note que vetor de categorias AY2 é tipo float32.

5. Classificar MNIST em Keras usando rede neural densa

5.1 Primeira rede densa para classificar MNIST

Agora, estamos prontos para escrever programa Keras que classifica MNIST usando rede neural densa. Programa 4 é essa implementação.

```
1 # mlp1.py
2 import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
3 import tensorflow.keras as keras
4 from tensorflow.keras.datasets import mnist
5 from tensorflow.keras.models import Sequential
6 from tensorflow.keras.layers import Dense, Flatten, Normalization
7 from tensorflow.keras import optimizers
8 import numpy as np; import sys
9
10 (AX, AY), (QX, QY) = mnist.load_data()
11 AX=255-AX; QX=255-QX
12
13 nclasses = 10
14 AY2 = keras.utils.to_categorical(AY, nclasses)
15 QY2 = keras.utils.to_categorical(QY, nclasses)
16
17 n1, nc = AX.shape[1], AX.shape[2] #28, 28
18
19 #1) Pseudo-normalizacao para intervalo [-0.5, +0.5]
20 #AX = (AX.astype('float32')/255.0)-0.5 # -0.5 a +0.5
21 #QX = (QX.astype('float32')/255.0)-0.5 # -0.5 a +0.5
22
23 #2) Normalizacao - distribuicao normal de media zero e desvio 1
24 #media=np.mean(AX); desvio=np.std(AX)
25 #AX=AX.astype('float32'); AX=AX-media; AX=AX/desvio; QX=QX.astype('float32'); QX=QX-media; QX=QX/desvio
26
27 #3) Normalizacao - inserir camada de normalizacao na rede
28 model = Sequential()
29 model.add(Normalization(input_shape=(n1,nc))) #Normaliza
30 model.add(Flatten())
31 model.add(Dense(400, activation='sigmoid'))
32 model.add(Dense(nclasses, activation='softmax'))
33
34 from tensorflow.keras.utils import plot_model
35 plot_model(model, to_file='mlp1.png', show_shapes=True)
36 model.summary()
37
38 opt=optimizers.Adam(learning_rate=0.0005)
39 model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
40
41 model.get_layer(index=0).adapt(AX) #Calcula media e desvio
42 model.fit(AX, AY2, batch_size=100, epochs=40, verbose=2)
43 score = model.evaluate(QX, QY2, verbose=0)
44 print('Test loss:', score[0])
45 print('Test accuracy:', score[1])
46 model.save('mlp1.keras')
```

Programa 4: Classificação de MNIST com rede neural densa.

<https://colab.research.google.com/drive/1cdA3TbL9OH1g7fDVzSFLUQFPry7zS4u7?usp=sharing> [~/deep/algpi/densa/mnist/mlp1.py]

Os números de 0 a 255 (uint8) não são adequados para alimentar uma rede neural. Fazendo isso, a rede demora para convergir ou não converge nunca. Para resolver isso, o programa apresenta 3 alternativas:

1) Podemos descomentar as linhas 19-21, e as matrizes AX e QX serão convertidas para o intervalo [-0.5, +0.5] (float32).

2) Outra possibilidade é normalizar subtraindo a média e dividindo pelo desvio, para que os valores sigam distribuição normal com média zero e desvio padrão um (linhas 23-25). O problema desta solução é que, quando salvar o modelo obtido num arquivo, é necessário armazenar em algum lugar a *média* e o *desvio* dos dados do treino AX, para usar os mesmos valores durante o teste.

Pergunta: Por que não se pode usar média e desvio dos dados de teste QX?

1) Pode ser que se queira classificar uma única imagem (ou poucas imagens) e neste caso não há como calcular média e desvio dos dados de teste.

2) Pode ser que nos dados de teste predomine algumas classes. Por exemplo, se há só dígitos “0” e “1” nos dados de teste, a média e o desvio serão diferentes dos dados de treino.

3) Em vez de efetuar normalização “manualmente”, pode-se usar uma camada Keras que efetua esta operação e armazena a média e desvio dentro da rede treinada como mais dois parâmetros da rede.

Esta é a solução que o programa usa. A linha 29 insere a camada de normalização e a linha 41 calcula a média e o desvio de AX antes de treinar a rede. A rede vai normalizar as entradas tanto durante o treino como durante o teste usando média e desvio dos dados de treino AX . A média e o desvio de AX ficarão armazenadas na rede *mlp1.keras*.

Nota: A figura 10 mostra que a normalização tem 57 parâmetros em vez de 2, pois consegue fazer mais tarefas do que simplesmente subtrair média e dividir por desvio-padrão. Veja [https://keras.io/api/layers/preprocessing_layers/numerical/normalization/]

As linhas 28-32 especificam o modelo da rede neural, com uma única camada escondida com 400 neurônios. Na primeira camada é necessário especificar o formato dos dados de entrada (*input_shape*).

A camada (“Flatten”, linha 30) é uma camada artificial que não faz nenhum processamento: só converte matriz 2D (None, 28, 28) para vetor 1D de (None, 784) elementos, pois a próxima camada (Dense) deve obrigatoriamente receber vetores 1D. A primeira dimensão “None” dos tensores refere-se ao índice dentro do lote.

A terceira camada (linha 31) é a camada escondida e possui 400 neurônios e ativação sigmoide.

A quarta camada (linha 32) é a camada de saída com 10 neurônios e ativação também sigmoide.

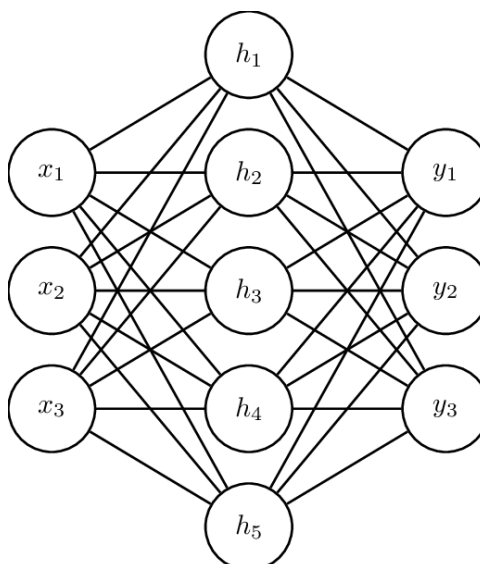


Figura 9: A rede do programa 4 recebe $28 \times 28 = 784$ dados de entrada (x), possui uma camada escondida com 400 neurônios (h) e uma camada de saída (y) com 10 neurônios (esta figura apresenta quantidades menores de neurônios).

As linhas 34-36 mostram duas diferentes formas de imprimir o modelo da rede. A saída do modelo impressa pelo programa, mostrando as 3 camadas da rede está abaixo:

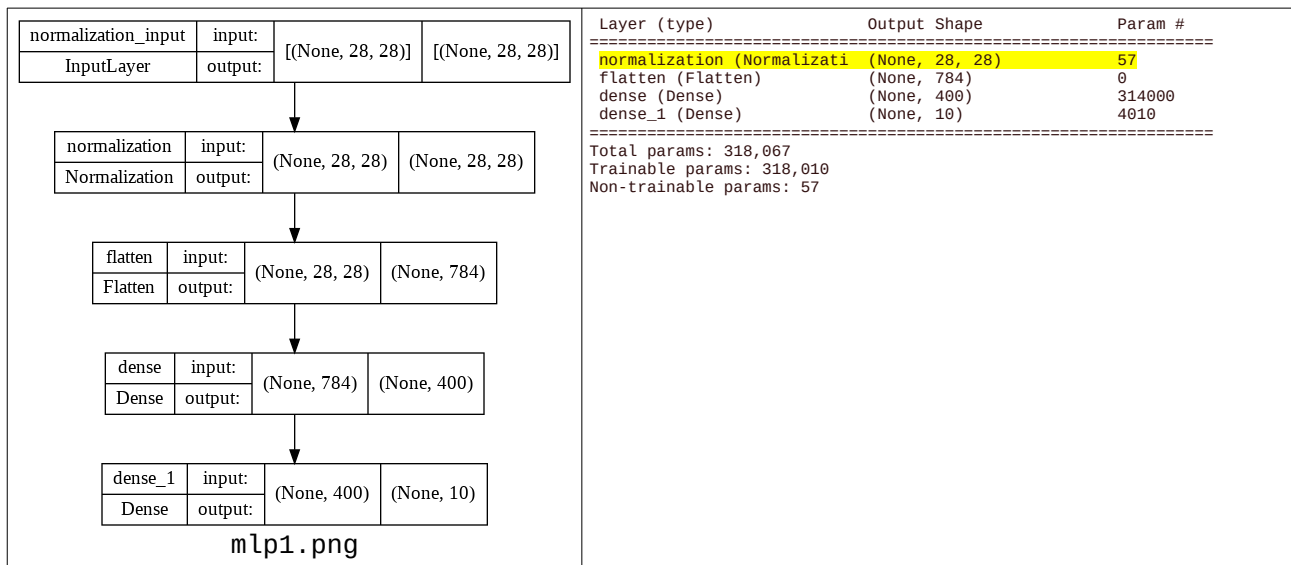


Figura 10: Duas formas de Keras mostrar a estrutura da rede.

As linhas 38-39 descrevem o otimizador (Adam), a função custo (MSE) e as métricas adicionais para exibir durante o treino (accuracy). O otimizador Adam é um otimizador melhor do que SGD e provavelmente é o mais usado na prática. Adam usa “momentum” e taxa de aprendizado adaptativo para convergir mais rapidamente à solução ótima. A função de erro continua sendo MSE e vamos imprimir também a acuracidade para melhor poder acompanhar o progresso da qualidade do modelo.

A linha 41 inicializa a camada de normalização, calculando a média e o desvio dos dados de treino. Não se esqueça de chamar *adapt* para calcular média e desvio se você colocou uma camada de normalização.

A linha 42 faz o treino. Tamanho de mini-batch é 100 (vai calcular gradiente e fazer back-propagation ajustando os parâmetros para cada lote de 100 amostras de treino), e vai rodar 40 épocas (todas as amostras de treino serão utilizadas 40 vezes). A saída obtida durante o treino é:

```
Epoch 1/40 - 1s - loss: 0.0265 - accuracy: 0.8728 - 1s/epoch - 2ms/step
Epoch 10/40 - 1s - loss: 0.0034 - accuracy: 0.9840 - 1s/epoch - 2ms/step
Epoch 20/40 - 1s - loss: 9.5126e-04 - accuracy: 0.9958 - 1s/epoch - 2ms/step
Epoch 30/40 - 1s - loss: 3.0444e-04 - accuracy: 0.9983 - 1s/epoch - 2ms/step
Epoch 40/40 - 1s - loss: 1.3329e-04 - accuracy: 0.9990 - 1s/epoch - 2ms/step
```

As funções custo e acuracidade listados acima são calculados nos dados de treino AX, AY. A função custo e acuracidade nos dados de teste são calculados e impressos nas linhas 43-45:

```
Test loss: 0.003645111806690693
Test accuracy: 0.9799000024795532
```

Isto é, a taxa de erro de teste obtida foi **2,0%**, uma das menores taxas obtidas para MNIST na aula “classif-ead” usando algoritmos clássicos (o único método com taxa de erro de teste abaixo de 2% foi SVM, com 1,95%). Esta taxa é menor do que aquela que obtivemos com vizinho mais próximo (1-NN). A taxa de erro de treino foi 0,01%.

Nota: Estes valores mudam a cada execução do programa, pois os pesos são inicializados aleatoriamente.

Por fim, a linha 46 salva o modelo de rede obtida. Este arquivo *mlp1.keras* permite fazer outros testes ou continuar o treino de onde parou.

5.2 Segunda rede densa para classificar MNIST

Vamos tentar melhorar a rede densa utilizando técnicas mais modernas. Já trocamos o otimizador clássico SGD (stochastic gradient descent) pelo otimizador moderno Adam (adaptive moment estimation). Vamos:

- 1) Colocar mais uma camada escondida.
- 2) Trocar função de ativação de camadas escondidas (sigmoide) pela função moderna relu (anexo A.2).
- 3) Trocar função de ativação da última camada (sigmoide) pela função softmax (anexo A.5).
- 4) Mudar a função de perda de MSE para categorical_crossentropy (anexo A.6).
- 5) Aumentar número de épocas de 40 para 80.

Essas alterações estão marcados em amarelo no programa 5.

```
1 # mlp2.py
2 import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
3 import tensorflow.keras as keras
4 from tensorflow.keras.datasets import mnist
5 from tensorflow.keras.models import Sequential
6 from tensorflow.keras.layers import Dense, Flatten, Normalization
7 from tensorflow.keras import optimizers
8 import numpy as np; import sys
9
10 (AX, AY), (QX, QY) = mnist.load_data()
11 AX=255-AX; QX=255-QX
12
13 nclasses = 10
14 AY2 = keras.utils.to_categorical(AY, nclasses)
15 QY2 = keras.utils.to_categorical(QY, nclasses)
16
17 nl, nc = AX.shape[1], AX.shape[2] #28, 28
18
19 model = Sequential()
20 model.add(Normalization(input_shape=(nl,nc))) #Normaliza
21 model.add(Flatten())
22 model.add(Dense(400, activation='relu'))
23 model.add(Dense(100, activation='relu'))
24 model.add(Dense(nclasses, activation='softmax'))
25
26 opt=optimizers.Adam(learning_rate=0.0005)
27 model.compile(optimizer=opt, loss='categorical_crossentropy',
28               metrics=['accuracy'])
29 model.get_layer(index=0).adapt(AX) #Calcula media e desvio
30 model.fit(AX, AY2, batch_size=100, epochs=80, verbose=2);
31
32 score = model.evaluate(QX, QY2, verbose=False)
33 print('Test loss: %.4f'%(score[0]))
34 print('Test accuracy: %.2f %'%(100*score[1]))
35 print('Test error: %.2f %'%(100*(1-score[1])))
36 model.save('mlp2.keras')
```

Programa 5: Classificação de MNIST com rede neural densa melhorada.

<https://colab.research.google.com/drive/1joGRic4qr66dcrpqU4VTCgAP8JC-iUEv?usp=sharing>

Executando o programa 5 (mlp2.py), obtemos:

```
Epoch 1/80 - 2s - loss: 0.2514 - accuracy: 0.9261 - 2s/epoch - 3ms/step
Epoch 20/80 - 1s - loss: 0.0060 - accuracy: 0.9979 - 1s/epoch - 2ms/step
Epoch 40/80 - 1s - loss: 0.0025 - accuracy: 0.9992 - 1s/epoch - 2ms/step
Epoch 60/80 - 1s - loss: 1.0177e-05 - accuracy: 1.0000 - 1s/epoch - 2ms/step
Epoch 80/80 - 1s - loss: 6.7866e-08 - accuracy: 1.0000 - 1s/epoch - 2ms/step
Test loss: 0.1706
Test accuracy: 98.39 %
Test error: 1.61 %
```

Onde o erro de treino é 0,00% (não errou nada) e o erro de teste é 1,61% (era 2,01% no programa anterior). Esta taxa de erro de teste é a menor já obtida até agora usando algoritmos clássicos. Porém, não conseguimos obter uma taxa de erro substancialmente menor que as que obtivemos até agora.

Nota: Executando novamente, às vezes obtenho test error 1.6% outras vezes 2.0%. Na verdade, precisa executar o programa várias vezes e tirar média.

Nota 1: Trocando ReLU por sigmoide no programa 5, obtive taxa de erro de teste 1,86%.

Nota 2: Usando rede neural convolucional, chegaremos a taxa de erro de menos de 0,4%.

Nota 3: Usando vision transformer chegaremos a taxa de erro de teste de 0,90%.

Exercício: Os resultados obtidos acima não são comparáveis aos dos métodos vistos na apostila “classif-ead”, pois lá trabalhamos com dígitos eliminando linhas/colunas brancas e redimensionado as imagens para 14×14 pixels. Modifique os programas acima para que trabalhem nas mesmas condições da aula “classificação” e compare as acurácias assim obtidas com as dos métodos daquela aula.

Nota: Fazendo isso para mlp1.py, obtive erro de 1,9%. Solução em ~/deep/algpi/densa/mnist/mlp3.py.

Exercício: Modifique os programas 4 ou 5 para obter taxa de erro de teste menor que 1,61%, sem usar rede neural convolucional. Algumas alterações possíveis são: (a) Acrescentar ou eliminar camadas. (b) Mudar o número de neurônios das camadas. (c) Mudar função de ativação. (d) Mudar o otimizador e/ou os seus parâmetros. (e) Mudar o tamanho do batch. (f) Mudar o número de épocas. (g) Eliminar linhas/colunas brancas das imagens de entrada. (h) Redimensionar as imagens de entrada. (i) Aumentar artificialmente os dados de treinamento, criando versões distorcidas das imagens. Descreva (como comentários dentro do seu programa .cpp ou .py) a taxa de erro que obteve, o tempo de processamento, as alterações feitas e os testes que fez para chegar ao seu programa com baixa taxa de erro.

Exercício: Use *SparseCategoricalCrossentropy* (anexo A.6.b2) para eliminar o cálculo *one-hot-encoding* (to_categorical) no programa 5.

[Solução privada em https://colab.research.google.com/drive/1cgBL9MkRPNeRzH9YQkzBvM0LYGpa7dm?usp=share_link]

Exercício: Troque as funções de ativação ReLU por outras, por exemplo, sigmoid, tanh, GELU, leaky relu, ELU, Swish, etc, e compare as taxas de acerto.

Exercício: Troque as funções de ativação ReLU por DAL (Dual Activation Layer)

(<https://doi.org/10.1109/ACCESS.2026.3666073>, http://www.lps.usp.br/hae/How_Does_Fourier_Analysis_Network_Work_Enhanced_Version.pdf)

e compare as taxas de acerto.

Solução: <https://colab.research.google.com/drive/1nkObK42NP1eZ-duZZ1bIU1pbLZofzNH>

A taxa de erro final não parece mudar.

5.3 Exemplo de carregar rede já treinada para fazer teste

O programa abaixo mostra como carregar uma rede já treinada para fazer testes. Uma rede já treinada também pode ser carregada para continuar o treino do ponto em que parou. Este programa deve ser executado após ter executado o programa 5 (mlp2.py).

```
1 #pred2.py
2 import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
3 import tensorflow.keras as keras
4 from tensorflow.keras.datasets import mnist
5 from tensorflow.keras.models import load_model
6 from tensorflow.keras.utils import to_categorical
7
8 (_, _), (QX, QY) = mnist.load_data()
9 QX=255-QX
10
11 nclasses = 10
12 QY2 = keras.utils.to_categorical(QY, nclasses)
13 nI, nc = QX.shape[1], QX.shape[2] #28, 28
14
15 model=load_model('mlp2.keras')
16 score = model.evaluate(QX, QY2, verbose=False)
17 print('Test loss: %.4f'%(score[0]))
18 print('Test accuracy: %.2f %%'%(100*score[1]))
19 print('Test error: %.2f %%'%(100*(1-score[1])))
20
21 QP2 = model.predict(QX); QP = QP2.argmax(axis=-1)
22 print(QP2[0])
23 print(QP[0])
24 #Aqui QP contem as 10000 predicoes
25 print("Imagem-teste 0: Rotulo verdadeiro=%d, predicacao=%d"%(QY[0], QP[0]))
26 print("Imagem-teste 1: Rotulo verdadeiro=%d, predicacao=%d"%(QY[1], QP[1]))
```

Programa 6: Lê modelo já treinado e faz predição.

<https://colab.research.google.com/drive/1joGRic4qr66dcrpqU4VTCgAP8JC-iUEv?usp=sharing> no segundo bloco de código.

Saída:

```
Test loss: 0.1653
Test accuracy: 98.24 %
Test error: 1.76 %
313/313 [=====] - 1s 2ms/step
[1.0139429e-24 9.3989936e-26 1.8574848e-25 1.7209503e-16 3.3879387e-24
 2.2341992e-22 3.2218744e-31 1.0000000e+00 1.3698717e-21 7.5743600e-15]
7
Imagem-teste 0: Rotulo verdadeiro=7, predicacao=7
Imagem-teste 1: Rotulo verdadeiro=2, predicacao=2
```

Exercício: Fashion_MNIST é um BD muito semelhante à MNIST. Consiste de 60000 imagens de treino e 10000 imagens de teste 28×28, em níveis de cinza, com as categorias:

categories=["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
categorias=["Camiseta", "Calça", "Pulôver", "Vestido", "Casaco", "Sandália", "Camisa", "Tênis", "Bolsa", "Botins"]

Esse BD pode ser carregada com os comandos:

```
from keras.datasets import fashion_mnist
```

```
(AX, AY), (QX, QY) = fashion_mnist.load_data()
```

Modifique um dos programas de rede neural densa que fizemos nesta aula para que classifique fashion_mnist (em vez de MNIST). Ajuste os parâmetros para atingir a menor taxa de erros. Qual foi a taxa de erro obtida? Deixe claro no vídeo e como comentário de programa a taxa de erro. Mostre as saídas das 20 primeiras imagens de teste com a classificação correta e a classificação dada pelo algoritmo. A figura abaixo mostra em azul a classificação verdadeira e em vermelho a classificação dada pelo algoritmo. Se vocês conseguirem deixar a saída mais “bonita”, melhor.



```
f = plt.figure()
for i in range(20):
    f.add_subplot(4,5,i+1)
    plt.imshow( [i-ésima imagem], cmap="gray")
    plt.axis("off");
    plt.text(0, -3, [i-ésima classificação verdadeira], color="b")
    plt.text(0, 2, [i-ésima classificação do algoritmo], color="r")
plt.savefig("nome_imagem.png")
plt.show()
```

Referências:

- [Mazur] <http://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>
- [Nielsen] <http://neuralnetworksanddeeplearning.com/>
- [WikiRelu] [https://en.wikipedia.org/wiki/Rectifier_\(neural_networks\)](https://en.wikipedia.org/wiki/Rectifier_(neural_networks))
- [Optimizers] <https://mlfromscratch.com/optimizers-explained/#/>
- [WikiSoftmax] https://en.wikipedia.org/wiki/Softmax_function

Usar Wisconsin breast cancer data
<https://www.kaggle.com/code/thebrownvikings20/intro-to-keras-with-breast-cancer-data-ann>
Usar TensorBoard

Anexo A: Definições matemáticas (funções de ativação e loss)

A.1 Inicialização dos pesos e vieses

Cada vez que roda um programa Keras, resulta numa taxa de erro um pouco diferente, pois os pesos são inicializados aleatoriamente.

O inicializador default de Keras e TensorFlow é (para maioria das camadas) `glorot_uniform` ou `Xavier uniform`.

Nota: O inicializador padrão de PyTorch é Kaiming/He, o que faz com que Keras e PyTorch tenham comportamentos diferentes.

Do manual do Keras:

```
keras.initializers.glorot_uniform(seed=None)  
Glorot uniform initializer, also called Xavier uniform initializer.
```

It draws samples from a uniform distribution within $[-limit, +limit]$ where $limit$ is $\sqrt{6 / (fan_in + fan_out)}$ where fan_in is the number of input units in the weight tensor and fan_out is the number of output units in the weight tensor.

Arguments

`seed`: A Python integer. Used to seed the random generator.

Exemplo:

```
model = Sequential([  
    Dense(3, input_shape=(4, ), kernel_initializer='glorot_uniform'), # Camada A  
    Dense(2, kernel_initializer='glorot_uniform')                     # Camada B  
])
```

Para a camada A, $limit = \sqrt{\frac{6}{4+3}}$, ou seja, os pesos são sorteados no intervalo $[-0.9258, +0.9258]$.

- $n_{in}=4$ (porque `input_shape=(4,)`)
- $n_{out}=3$ (porque `units=3`)

Para a camada B, $limit = \sqrt{\frac{6}{3+2}}$, ou seja, os pesos são sorteados no intervalo $[-1.0954, +1.0954]$.

- $n_{in}=3$ (vem da camada anterior)
- $n_{out}=2$

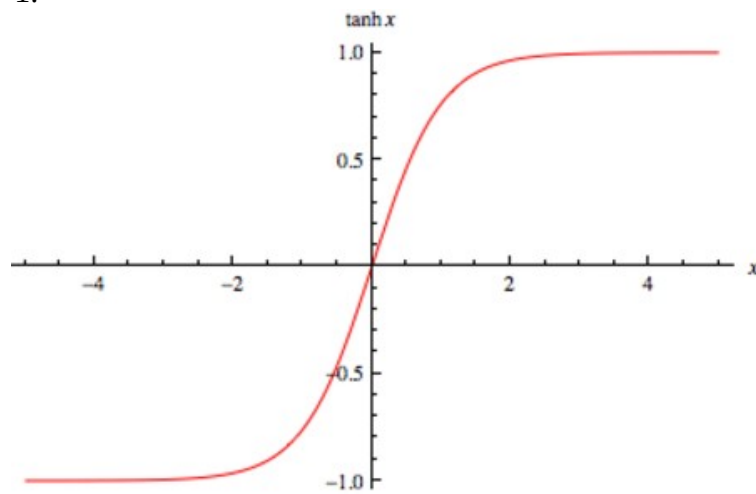
Os vieses são inicializados com zero.

A inicialização `glorot_uniform` foi projetada para quando os dados de entrada têm distribuição normal com média zero e desvio padrão igual a 1.

A.2 Função de ativação tangente hiperbólico

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

A saída vai de -1 a +1.



Keras/Tensorflow

```
# tanh.py
import tensorflow as tf
import numpy as np
a = tf.constant([-4, -2, 0, 2, 4], dtype = tf.float32)
b = tf.keras.activations.tanh(a)
print(b.numpy())
```

Saída:

```
[-0.9993292 -0.9640276  0.          0.9640276  0.9993292]
```

A sua derivada é:

$$\tanh'(x) = \text{sech}^2(x) = 1/\cosh^2(x)$$

A.3 Função de ativação sigmoide ou logistic

A função de ativação *sigmoide* ou *logistic* é definida como:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

e o seu gráfico está na figura abaixo. Note que a saída vai de 0 a 1.

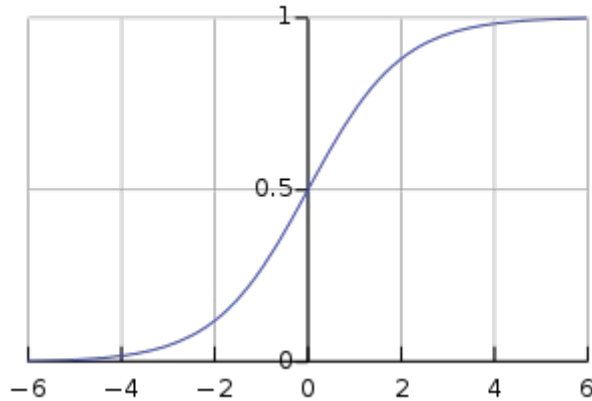


Figura: Função logística sigmoide [extraída da Wikipedia].

Keras/Tensorflow:

```
# sigmoid.py
import tensorflow as tf
import numpy as np
a = tf.constant([-4, -2, 0, 2, 4], dtype = tf.float32)
b = tf.keras.activations.sigmoid(a)
print(b.numpy())
```

Saída:

```
[0.01798621 0.11920292 0.5          0.880797   0.98201376]
```

A função inversa de *sigmoide* chama-se *logit*:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

A derivada da sigmoide é:

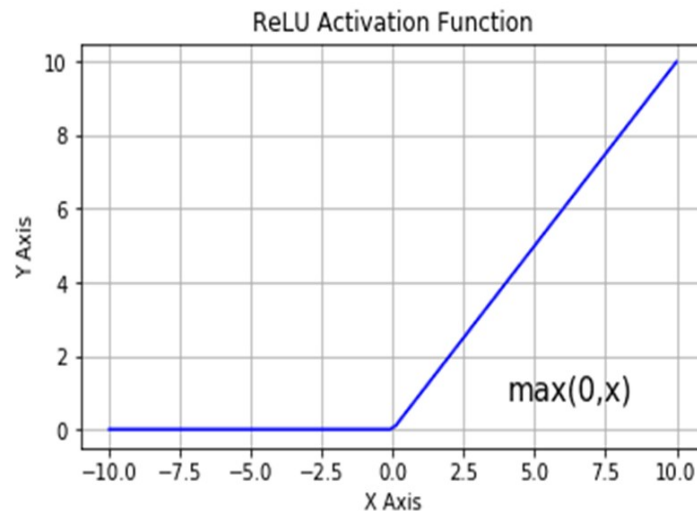
$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

A.4 Função de ativação ReLU

ReLU (rectified linear unit) é uma função de ativação introduzida no ano 2000 e é a função de ativação mais popular em 2017 [WikiRelu]. Foi demonstrada em 2011 que permite melhor treinamento de redes profundas. A sua definição é:

$$f(x) = \max(0, x)$$

Parece a função de transferência $V_{in} \times V_{out}$ de um diodo ideal.



Curva relu (rectified linear unit).

A grande vantagem desta função de ativação é a propagação eficiente de gradiente: minimiza problema de “vanishing or exploding gradient” (veja mais detalhes no livro [Nielsen]). Note que a derivada da relu é zero ou um, o que ajuda a evitar que gradiente exploda ou desapareça.

Tensorflow/Keras

```
# relu.py
import tensorflow as tf
import numpy as np
a = tf.constant([-4, -2, 0, 2, 4], dtype = tf.float32)
b = tf.keras.activations.relu(a)
print(b.numpy())
```

Saída:

[0. 0. 0. 2. 4.]

A derivada da relu $r(x)$ é:

$$r'(x) = \begin{cases} 0 & \text{se } x < 0 \\ 1 & \text{se } x > 0 \\ \text{indefinido} & \text{se } x = 0 \end{cases}$$

A.5 Funções de ativação “semelhantes a ReLU”:

Depois de ReLU, foram propostas várias funções de ativação inspiradas em ReLU, para minimizar os defeitos de ReLU (principalmente “dead neurons”):

- Piecewise Linear: ReLU, Leaky ReLU, PReLU.
- Smooth Alternatives: ELU, SELU, Swish, GELU.
- Avoid “Dying Neurons”: Leaky ReLU, PReLU, ELU, SELU.

A.6 Softmax

A função *softmax*, também conhecido como *softargmax* ou *normalized exponential function*, é uma função que pega como entrada um vetor de K números reais e normaliza-o em distribuição de probabilidade com K probabilidades. Após aplicar a função *softmax*, cada componente do vetor estará no intervalo $(0,1)$ e a soma dos componentes será 1, de forma que eles podem ser interpretados como probabilidades. Além disso, componentes de entrada maiores irão corresponder a probabilidades maiores.

A função softmax padrão $\text{softmax}:\mathbb{R}^K \rightarrow \mathbb{R}^K$ é definida como:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Exemplo numérico:

Vetor de entrada $\mathbf{z}=(-0.1, 1.3, 0.3)$

Calculando a soma dos exp's: $\sum_{j=1}^K e^{z_j} = e^{-0.1} + e^{1.3} + e^{0.2} = 5.7955$

Calculando as saídas:

$$\text{softmax}(\mathbf{z})_1 = \frac{e^{-0.1}}{5.7955} = 0.15613$$

$$\text{softmax}(\mathbf{z})_2 = \frac{e^{1.3}}{5.7955} = 0.63313$$

$$\text{softmax}(\mathbf{z})_3 = \frac{e^{0.2}}{5.7955} = 0.21075$$

Assim:

$$\text{softmax}(-0.1, 1.3, 0.3) = (0.15613, 0.63313, 0.21075)$$

Keras:

```
# softmax.py
import tensorflow as tf
import numpy as np
inp = np.asarray([-0.1, 1.3, 0.2])
layer = tf.keras.layers.Softmax()
print(layer(inp).numpy())
```

Saída:

[0.1561266 0.6331246 0.21074888]

Softmax pode levar diferentes entradas em saídas iguais e portanto não é possível calcular a função inversa (a não ser que se disponha de mais informações).

A.7 Cross-entropy loss:

Para entender cross-entropy loss, vou seguir a explicação de:

https://gombu.github.io/2018/05/23/cross_entropy_loss/

https://ml-cheatsheet.readthedocs.io/en/latest/loss_functions.html

As funções de perda em Keras:

<https://neptune.ai/blog/keras-loss-functions>

a) Problemas multi-classe, multi-rótulo e binário:

Primeiro, vamos distinguir três tipos de problemas de classificação: *multi-class*, *multi-label* e binário.

1) Uma classificação multi-classe consiste em categorizar uma instância em uma de um conjunto classes possíveis. Por exemplo, classificar um dígito manuscrito MNIST nas classes '0' a '9'.

2) Um problema multi-rótulo consiste em atribuir a uma instância um ou mais rótulos. Por exemplo, classificar um indivíduo em jovem/idoso, homem/mulher e usa/não usa óculos.

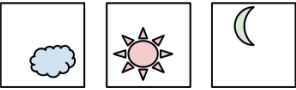
	Multi-Class	Multi-Label
C = 3	Samples  Labels (t) [0 0 1] [1 0 0] [0 1 0]	Samples  Labels (t) [1 0 1] [0 1 0] [1 1 1]

Figura: Diferença entre problema multi-class e multi-label

[https://gombu.github.io/2018/05/23/cross_entropy_loss/]

3) Uma classificação binária ou booleana consiste em classificar instâncias em 2 grupos, por exemplo: cachorro ou gato, câncer ou não-câncer, etc. Uma rede neural que resolve este tipo de problema pode gerar na saída um vetor com dois elementos (por exemplo, p_0 = probabilidade de ser cachorro, p_1 = probabilidade de ser gato) ou uma única saída p (por exemplo, quanto mais p for próximo de 1 for a saída, mais provável que a imagem seja de gato). No primeiro caso, este problema é um problema multi-classe de 2 classes. No segundo caso, este problema é um problema multi-rótulo com apenas um rótulo.

b) Categorical cross-entropy loss (para problemas multi-classe):

Categorical cross-entropy é usada na classificação multi-classe (e na classificação binária em redes com 2 saídas).

Normalmente, a função *softmax* é aplicada no vetor das previsões p antes de calcular *categorical cross-entropy*, para que os elementos do vetor p se comportem como probabilidades. Neste caso, chame *CategoricalCrossentropy* com parâmetro *from_logits=False* (default). Se *softmax* não foi aplicada em p , é possível pedir que *CategoricalCrossentropy* aplique-a antes de calcular a função de perda colocando parâmetro *from_logits=True*.

Nota: O que é *logits*? Em aprendizado de máquina, *logits* indica as previsões antes de passar pela normalização *softmax* [<https://stackoverflow.com/questions/41455101/what-is-the-meaning-of-the-word-logits-in-tensorflow>]. Também pode significar a função inversa da *sigmoide*.

Nota: A recomendação de Keras é, para maior estabilidade numérica, não calcular *softmax* e chamar *CategoricalCrossentropy* com *from_logits=True*.

Categorical cross-entropy é definida como:

$$L(p, y) = - \sum_{i=1}^K \log(p_i) y_i, \text{ sendo } \log \text{ natural (base } e\text{).}$$

onde p_i e y_i são respectivamente saída fornecida pela rede no intervalo $[0, 1]$ e o rótulo verdadeiro $\{0, 1\}$ para uma certa entrada x e para cada classe i de K .

O vetor y contém um único componente 1, sendo o resto 0 (chamado “one-hot” labels, tanto em “multi-class classification” como em “binary classification” com 2 saídas). Assim, a equação acima reduz-se a:

$$L(p, y) = (-\log p_i)_{y_i=1}.$$

Tensorflow/Keras possui duas funções para categorical cross-entropy:

Exemplo b1) *CategoricalCrossentropy* calcula a perda entre o vetor de previsões da rede e one-hot encoding:

$$p=(0.1, 0.8, 0.1) \quad y=(0, 1, 0) \quad L(p,y) = -\log(0.8) = 0.22314$$

```
# cce.py - CategoricalCrossentropy
import tensorflow as tf
import numpy as np
y_true = [0, 1, 0]
y_pred = [0.1, 0.8, 0.1]
bce = tf.keras.losses.CategoricalCrossentropy(from_logits=False)(y_true, y_pred)
print(bce.numpy())
```

Saída: 0.22314353

Esta é a diferença entre (0.1, 0.8, 0.1) e (0, 1, 0) calculado usando categorical cross entropy.

Exemplo b2) *SparseCategoricalCrossentropy* calcula a perda entre o vetor de previsões da rede e o rótulo verdadeiro:

$$p=(0.1, 0.8, 0.1) \quad y=1 \quad L(p,y) = -\log(y_1) = -\log(0.8) = 0.22314$$

```
# scce.py - SparseCategoricalCrossentropy
import tensorflow as tf
import numpy as np
y_true = 1 #A categoria verdadeira e' 1
y_pred = [0.1, 0.8, 0.1]
bce = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False)(y_true, y_pred)
print(bce.numpy())
```

Saída: 0.22314355

Estamos indicando, com `from_logits=False`, que já aplicamos *softmax* no vetor de predições p . Colocaríamos `from_logits=True` para indicar que não aplicamos *softmax*.

Quando o número de classes K é dois, o problema torna-se uma classificação binária. Para usar categorical cross-entropy, é necessário que a rede tenha 2 saídas.

c) Binary cross-entropy loss (para problemas multi-rótulo):

Binary cross-entropy é usada para “multi-label classification” (e classificação binária em redes com uma única saída).

Antes de aplicar binary cross-entropy, muitas vezes função sigmoide é aplicada em cada elemento do vetor de predições p para que se comporte como probabilidade.

[<https://medium.com/ensina-ai/uma-explica%C3%A7%C3%A3o-visual-para-fun%C3%A7%C3%A3o-de-custo-binary-cross-entropy-ou-log-loss-eaee662c396c>]

Binary cross-entropy para classificação multi-label com K classes é definida como:

$$L(p, y) = -\frac{1}{K} \left(\sum_{i=1}^K y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right)$$

Considerando que os rótulos y_i são sempre ou 0 ou 1, essa equação pode ser reescrita como:

$$L(p, y) = -\frac{1}{K} \left(\sum_{y_i=1} \log(p_i) + \sum_{y_i=0} \log(1 - p_i) \right)$$

Exemplo c1) Binary Cross Entropy para multi-label classification:

Exemplo:

$p=(0.4, 0.8, 0.6)$ $y=(0, 1, 1)$

$$L(p, y) = (-1/3) * (\log(1-0.4) + \log(0.8) + \log(0.6)) = (-1/3) * (-0.51083 -0.22314 -0.51083) = 0.41493$$

```
# bce.py - BinaryCrossentropy
import tensorflow as tf
import numpy as np
y_true = [0, 1, 1]
y_pred = [0.4, 0.8, 0.6]
bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)(y_true, y_pred)
print(bce.numpy())
```

Saída: 0.4149314

Esta é a diferença entre (0.4, 0.8, 0.6) e (0, 1, 1) calculada usando binary cross entropy.

Exemplo c2) Binary Cross Entropy para classificação binária:

Exemplo – classificar em cachorro=0 ou gato=1.

$p=0.4$ $y=0$

$$L(p, y) = \log(1-0.4) = 0.51083$$

```
# bce.py - BinaryCrossentropy
import tensorflow as tf
import numpy as np
y_true = [0, 1, 1]
y_pred = [0.4, 0.8, 0.6]
bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)(y_true, y_pred)
print(bce.numpy())
```

Saída: 0.5108254

Estamos indicando, com `from_logits=False`, que já aplicamos *sigmoid* em cada elemento do vetor de predições p .

Exemplo de multi-label classification: Considere que um programa identifica se numa imagem há avião, navio, automóvel e bicicleta. Dada uma imagem x , o programa devolveu a saída $s=[-2, -1, 2, 3]$, indicando que é pouco provável que haja avião e navio na imagem, mas é provável que haja automóvel e bicicleta. Os elementos deste vetor devem ser convertidos em números entre 0 e 1, para que se tornem espécie de “probabilidades”. Para isso, utiliza-se a função sigmoide:

$$p = \text{sigmoide}([-2, -1, 2, 3]) = [0.119, 0.269, 0.881, 0.9526]$$

Suponha que os rótulos verdadeiros são $y=[0, 0, 1, 1]$. Agora, é possível aplicar *binary cross-entropy loss* a cada elemento de p :

$$\text{BCE}(p, y) = [0.1269, 0.3133, 0.1269, 0.0486]$$

Calculando a média, obtemos:

$$L(p, y) = 0.1539$$

Exemplo em Keras:

```
import os; os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
import keras; import numpy as np
s = np.array([-2, -1, 2, 3], dtype=np.float32)
y = np.array([0, 0, 1, 1], dtype=np.float32)
p = keras.activations.sigmoid(s); print("p=", p.numpy())
loss = keras.losses.BinaryCrossentropy(from_logits=False)
L = loss(y, p); print("L=", L.numpy())
loss = keras.losses.BinaryCrossentropy(from_logits=True)
L = loss(y, s); print("L=", L.numpy())
```

Saída:

p= [0.11920292 0.26894143 0.880797 0.95257413]

L= 0.15392613

L= 0.15392627

[Nota para mim: Acrescentar Accuracy, CategoricalAccuracy e SparseCategoricalAccuracy.]

Anexo B: Outras dicas sobre Google Colab

Fazer download de arquivos de internet para Google Colab

a) É possível fazer download de um arquivo da internet e gravar no diretório de Colab usando o comando `wget` [<https://linuxize.com/post/wget-command-examples/>]:

```
!wget [opções] url
```

O comando acima faz download do arquivo especificado em *url* no diretório atual. Por exemplo:

```
!wget https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png
```

baixa imagem *Lenna_(test_image).png* do site Wikipedia. Vários sites, incluindo muitos sites da USP, bloqueiam download via `wget`. É possível contornar este problema especificando a opção “--user-agent” ou “-U”, emulando a requisição por um aplicativo diferente. O comando abaixo emula a requisição de download por Firefox 50.0, contornando o bloqueio:

```
!wget -nc -U 'Firefox/50.0' 'http://www.lps.usp.br/hae/apostila/hummingbird.jpg'
```

A opção “-nc” baixa o arquivo somente se precisar. O programa abaixo descarrega a imagem *hummingbird.jpg* do meu site no diretório /content do Colab (se a imagem já não estiver lá) e mostra-a na tela. Pode ser executado do Colab ou de bash:

```
#Funciona chamando python3 em bash ou Colab
url="http://www.lps.usp.br/hae/apostila/hummingbird.jpg"
import os; nomeArq=os.path.split(url)[1]
if not os.path.exists(nomeArq):
    os.system("wget -nc -U 'Firefox/50.0' "+url)
from matplotlib import pyplot as plt
a=plt.imread(nomeArq)
plt.imshow(a); plt.axis("off"); plt.show()
```

b) Normalmente, é muito mais rápido transferir um arquivo grande do que vários arquivos pequenos. Assim, se você precisa transferir um banco de dados com muitas imagens, é melhor transferir o arquivo compactado (por exemplo, como .zip) e descompactar no Colab. O programa abaixo transfere BD *feiCorCorp* com 400 imagens compactadas para diretório local da máquina Colab e o descompacta.

```
url='http://www.lps.usp.br/hae/apostila/feiCorCrop.zip'
import os; nomeArq=os.path.split(url)[1]
if not os.path.exists(nomeArq):
    print("Baixando o arquivo",nomeArq,"para diretorio default",os.getcwd())
    os.system("wget -nc -U 'Firefox/50.0' "+url)
else:
    print("O arquivo",nomeArq,"ja existe no diretorio default",os.getcwd())
print("Descompactando arquivos novos de",nomeArq)
os.system("unzip -u "+nomeArq)
```


c) Se você precisa trabalhar com base de dados (BD) com muitos arquivos em Colab, ficam lentas ambas as opções a seguir: (1) baixar BD da internet frequentemente; (b) transferir muitos arquivos do Google Drive para diretório local da máquina Colab. A solução é compactar todos os arquivos num grande arquivo BD.zip e deixá-lo no Google Drive. Toda vez que precisar, transfere-se BD.zip do Google Drive para um diretório local da máquina Colab e o descompacta:

```
# Montar drive Colab
from google.colab import drive
drive.mount('/content/drive')

# criar diretorio local na maquina Colab
!mkdir my_dir
%cd my_dir/

# Copiar BD
!cp '/content/drive/MyDrive/BD.zip' .

# Descompactar BD
!unzip BD.zip
```

d) Você pode baixar arquivo do Google Drive para Colab ou para computador local sem ter que montar drive. Para isso, use “gdown”.

Nota: Esta técnica só funciona para baixar arquivos com permissão “Qualquer pessoa na Internet com o link pode ver.”

No computador local, é preciso instalar este módulo:

```
pip install gdown OU
pip3 install gdown
```

No Colab, esse programa já está instalado.

Depois, digamos que queira fazer download do arquivo lenna.jpg de link:

```
https://drive.google.com/file/d/1AsE1VMFQN5vc0y0Tc5E1ZHPSnIWxocKq/view?usp=sharing
```

Para isso, dê o seguinte comando:

```
Computador_local$ gdown --id 1AsE1VMFQN5vc0y0Tc5E1ZHPSnIWxocKq OU
Computador_local$ gdown https://drive.google.com/uc?id=1AsE1VMFQN5vc0y0Tc5E1ZHPSnIWxocKq
Google_Colab$ !gdown --id 1AsE1VMFQN5vc0y0Tc5E1ZHPSnIWxocKq OU
Google_Colab$ !gdown https://drive.google.com/uc?id=1AsE1VMFQN5vc0y0Tc5E1ZHPSnIWxocKq
```

e) Digamos que queira fazer download do arquivo zip de Google Drive com link e descompactá-lo no diretório default:

<https://drive.google.com/file/d/XXXXXXXXXXXXXXXXXXXXXXXXXXXXX/view?usp=sharing>

Para isso:

```
import os; import gdown
nomeArq="nomearq.zip"
if not os.path.exists(nomeArq):
    !gdown --id XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
!unzip -u resnet_transf
```

Ou, em Python:

```
import os; import gdown
#url="https://drive.google.com/file/d/1LrJz-IhAixFnFtcaFh3H8v3G69Dhoodn"
id="1LrJz-IhAixFnFtcaFh3H8v3G69Dhoodn"
nomeArq="nomearq.ext"
if not os.path.exists(nomeArq):
    print("Baixando o arquivo", nomeArq, "para diretório default", os.getcwd())
    gdown.download(id=id, output=nomeArq)
else:
    print("O arquivo", nomeArq, "já existe no diretório default", os.getcwd())
```

f) Para baixar um arquivo de Colab para seu computador local:

<https://predictivehacks.com/?all-tips=how-to-download-files-and-folders-from-colab>

```
from google.colab import files
files.download('arquivo.ext')
```

Se quiser baixar vários arquivos ou um diretório inteiro, primeiro crie um ZIP e baixe ZIP:

```
#Zipa arquivos cnn*.png como cnn.zip e faz download no computador local
!zip cnn.zip cnn*.png
from google.colab import files
files.download('cnn.zip')
```

g) Para baixar um arquivo de Google Colab com acesso “restrito” mas ao qual você tem acesso por ser uma das “pessoas com acesso”. A forma de fazer isso está em:

<https://towardsdatascience.com/different-ways-to-connect-google-drive-to-a-google-colab-notebook-pt-1-de03433d2f7a>

Infelizmente esta receita não funciona para arquivos muito grandes (OverflowError: signed integer is greater than maximum). Funciona para arquivos de menos de 1 GB.

Copio abaixo a fórmula:

g1) Rode a célula abaixo:

```
from pydrive.auth import GoogleAuth
from google.colab import drive
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials
import pandas as pd
```

g2) Rode a célula, depois preencher corretamente `google_drive_file_id` e `nome_arquivo.ext`.

```
auth.authenticate_user()
gauth = GoogleAuth()
gauth.credentials = GoogleCredentials.get_application_default()
drive = GoogleDrive(gauth)
```

```
file_id = 'google_drive_file_id'
download = drive.CreateFile({'id': file_id})
download.GetContentFile('nome_arquivo.ext')
```

Onde google_drive_file_id é o link do arquivo google drive. Você consegue esta informação clicando o botão direita do mouse em cima do arquivo google drive, depois clicar em “gerar link” e “copiar link”. Fazendo isso, obtém algo como:

https://drive.google.com/file/d/4XJ3PcBKMJoP8LI8xHXZD_kIzsd-j5bv/view?usp=share_link

Desse link, você deve copiar somente a parte amarela.

Você deve substituir nome_arquivo.ext pelo nome do arquivo com que gostaria que o arquivo seja salvo.

h) Fazer download de um dataset da Kaggle

O site abaixo descreve como baixar um dataset da Kaggle no Colab ou no seu computador local:

<https://www.geeksforgeeks.org/how-to-import-kaggle-datasets-directly-into-google-colab/>

Resumindo as informações desse site, você primeiro precisa criar uma conta na Kaggle:

<https://www.kaggle.com/>

Depois de criar a conta, na página principal da Kaggle, clique em “Account” que levará à página “Settings”. Nessa página, clique em “Create New Token”. Isso baixará um arquivo “kaggle.json”, onde você obterá o seu *username* e uma *key*, algo como:

```
{"username": "seunome", "key": "9e9f84b521cd43657418455fce99120b"}
```

Agora, no seu ambiente Colab, copie o conteúdo da célula 1.

```
#https://www.geeksforgeeks.org/how-to-import-kaggle-datasets-directly-into-google-colab/
!pip3 install -q kaggle
!mkdir ~/.kaggle
!echo '{"username": "seunome", "key": "9e9f84b521cd43657418455fce99120b"}' > ~/.kaggle/kaggle.json
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download anasmohammedtahir/covidqu
!unzip -u covidqu.zip
```

Célula 1: Célula para baixar o BD da Kaggle. Substitua parte amarela com as suas informações.

Você deve substituir o conteúdo em amarelo pelas suas informações obtidas de “kaggle.json”. A execução dessa célula irá baixar o BD no seu diretório default Colab (/content) e o descompactará.

i) Desconectar o ambiente de execução

```
from google.colab import runtime
runtime.unassign()
```

[PSI3471 aula 10. Fim.]